

C++ Programlama Dili

Kapsamlı Uygulamalı Rehber

Temellerden Modern C++'a: Nesne Yönelimi, Bellek Yönetimi, Şablonlar ve STL

Bu belge, C++'ı temel sözdiziminden başlayarak nesne yönelimli programlamaya, bellek ve kaynak yönetiminin inceliklerine, şablonlara, STL'e ve modern C++17/20 özelliklerine kadar **derinlemesine ve çalışan kod örnekleriyle** anlatır. Her sınıfın kalbi olan **özel üye fonksiyonlar** (Beşin Kuralı), akıllı işaretçiler, istisna güvenliği ve operatör aşırı yükleme gibi kritik konular ayrıntılı biçimde ele alınır.

K1 (Temeller): program yapısı • tipler & auto • kontrol akışı • fonksiyonlar. **K2 (OOP):** sınıflar • kalıtım • polimorfizm & sanal fonksiyonlar.

K3 (Bellek & İşaretçiler): yığın/öbek • new/delete • işaretçi vs referans • const ile işaretçiler. **K4 (RAII & Akıllı İşaretçiler):** unique_ptr • shared_ptr • weak_ptr.

K5 (Beşin Kuralı): lvalue/rvalue & std::move • yıkıcı • kopya/taşıma yapıcı & atama • copy-and-swap. **K6 (Şablonlar):** fonksiyon/sınıf şablonları • variadic • özelleştirme.

K7 (STL): konteynerler • iteratörler • algoritmalar. **K8 (Lambda):** lambda ifadeleri • std::function. **K9 (İstisnalar):** try/catch • istisna güvenliği.

K10 (Anahtar Kelimeler): explicit • constexpr • noexcept • static/mutable/friend.

K11 (Operatörler): aritmetik • <=> • [], () • dönüşüm. **K12 (Modern C++):** yapısal bağlama • concepts & ranges.

Hazırlanma Tarihi: 19 Temmuz 2026

C++17 / C++20 • g++ / clang++ • -Wall -Wextra

Contents

1 Bir C++ Programının Yapısı	2
1.1 Başlık Dosyaları ve Ad Alanları (Namespaces)	2
2 Temel Tipler, Değişkenler ve auto	3
3 Kontrol Akışı	4
4 Fonksiyonlar	5
5 Sınıflar ve Nesneler	7
6 Kalıtım (Inheritance)	8
7 Polimorfizm ve Sanal Fonksiyonlar	9
8 Programın Bellek Düzeni	11
9 new ve delete: Öbek Belleği Yönetimi	12
9.1 Bellek Sızıntıları ve İkili Silme	12
10 İşaretçiler ve Referanslar	13
10.1 const ve İşaretçiler	14
10.2 Boş, Sarkan ve Yaban İşaretçiler	14
11 RAII: Kaynak Edinimi Başlatmadır	16
12 std::unique_ptr: Tekil Sahiplik	17
12.1 Özel Silici (Custom Deleter)	18
13 std::shared_ptr ve std::weak_ptr: Paylaşımlı Sahiplik	18
13.1 Döngüsel Referans Sorunu ve weak_ptr	19
14 Değer Kategorileri: lvalue ve rvalue	21
15 Yıkıcı, Kopya ve Taşıma — Beş Fonksiyon Bir Arada	22
15.1 Beşi Bir Arada: Tam Çalışan Örnek	22
15.2 1. Yıkıcı (Destructor)	24
15.3 2. Kopya Yapıcı (Copy Constructor)	24
15.4 3. Kopya Atama Operatörü (Copy Assignment)	24
15.5 4. Taşıma Yapıcı (Move Constructor)	25
15.6 5. Taşıma Atama Operatörü (Move Assignment)	25
16 Üç, Beş ve Sıfır Kuralı	25
16.1 Üçün Kuralı (Rule of Three) — C++98	25
16.2 Beşin Kuralı (Rule of Five) — C++11	25
16.3 Sıfırın Kuralı (Rule of Zero) — En İyi Yaklaşım	26
16.4 = default ve = delete ile Açık Kontrol	26
17 Copy-and-Swap Deyimi	27
18 Fonksiyon Şablonları	29
19 Sınıf Şablonları	30

20 Değişken Sayıda Şablon (Variadic) ve Kat İfadeleri	30
21 Standart Konteynerler	32
22 İteratörler	33
23 Algoritmalar	34
24 Lambda İfadeleri	36
25 std::function ve Fonksiyon Nesneleri	37
26 try, catch ve throw	39
27 Özel İstisnalar ve İstisna Güvenliği	40
28 explicit: İstenmeyen Dönüşümleri Engelleme	42
29 const ve constexpr	42
30 noexcept: İstisna Atmama Garantisi	43
31 static, mutable ve friend	44
31.1 static Üyeler	44
31.2 mutable Üyeler	44
31.3 friend Fonksiyonlar	45
32 Operatör Aşırı Yüklemenin Temelleri	46
33 Karşılaştırma ve Uzak Gemisi Operatörü (C++20)	47
34 Özel Operatörler: [], (), ve Dönüşüm	48
34.1 Alt Simge Operatörü []	48
34.2 Fonksiyon Çağrı Operatörü () — Fonksiyon Nesneleri	48
34.3 Dönüşüm Operatörü	49
35 Gereksiz Kopyalar ve Yeniden Tahsisler	51
35.1 Yanlış: reserve'süz büyüyen vektör	51
35.2 Doğru: kapasiteyi önceden ayır (reserve) + emplace	51
35.3 push_back vs emplace_back	52
35.4 Yanlış: döngüde gizli kopya (range-for)	52
35.5 Doğru: sabit referansla gez, kopyayı kaldır	53
36 String'ler: Tahsis, Birleştirme ve string_view	53
36.1 Yanlış: operator+ zinciriyle birleştirme	53
36.2 Doğru: reserve + += ile yerinde ekle	53
36.3 Yanlış: salt okunur parametre için gereksiz kopya/tahsis	54
36.4 Doğru: string_view ile sahiplenmeden, tahsissiz oku	54
36.5 Küçük String Optimizasyonu (SSO)	55
37 Taşıma Semantiğini Kaçırmak ve Yanlış std::move Kullanımı	55
37.1 Yanlış: dönüşte std::move (RVO'yu bozar)	55
37.2 Doğru: yerel değişkeni olduğu gibi döndür	55
37.3 Yanlış / Doğru: değere alıp saklarken	56

38 Bellek Düzeni ve Önbellek Dostu Kod	56
38.1 Yanlış: dağınık düğümler (list / vector-of-pointers)	56
38.2 Doğru: bitişik dizi (contiguous vector)	57
38.3 AoS ve SoA: sıcak/soğuk alan ayrımı	57
38.4 Yapı Hizalama ve Dolgu (Padding)	58
39 Tahsis Maliyetleri: shared_ptr, make_shared ve Nesne Havuzları	59
39.1 Yanlış: iki tahsis (shared_ptr + new)	59
39.2 Doğru: make_shared (tek tahsis)	59
39.3 Yanlış: sıcak döngüde binlerce küçük tahsis	59
39.4 Doğru: tamponu yeniden kullan (havuz / arena)	60
40 İşaretçi ve Kaynak Yönetimi: Klasik Hatalar	61
40.1 Yanlış: yerel değişkenin adresini döndürmek	61
40.2 Doğru: değeri döndür (ya da ömrü çağırana ait olsun)	61
40.3 Yanlış: yeniden tahsis iteratörü/işaretçiyi geçersiz kılar	62
40.4 Doğru: yeniden tahsisi önle veya indeksle çalış	62
40.5 Yanlış: ham işaretçiyle belirsiz sahiplik (çift silme / sızıntı)	63
40.6 Doğru: sahipliği unique_ptr ile tek ve net yap	63
41 Beşin Kuralı: Kaynak Yöneten Sınıfı Doğru Yazmak	63
41.1 Yanlış: yalnızca yıkıcı (Üçün Kuralı ihlali)	64
41.2 Doğru: beş özel fonksiyonu da tanımla	64
41.3 Doğru (daha iyisi): Sıfırın Kuralı	65
42 Özet: Bellek Optimizasyonu Kontrol Listesi	66
43 Yapısal Bağlama, if-init ve Yararlı Tipler	68
44 Concepts: Şablonlara Kısıt (C++20)	69
45 Ranges: Bileştirilebilir Algoritmalar (C++20)	70
46 Kapanış ve Altın Kurallar	70

KISIM 1

C++ Temelleri

1. Bir C++ Programının Yapısı

C++, C dilinin üzerine inşa edilmiş, çok paradigmalı (yordamsal, nesne yönelimli, jenerik ve fonksiyonel) bir sistem programlama dilidir. Derlenen (compiled) bir dildir: kaynak kod, doğrudan makine koduna çevrilir; bu da onu son derece hızlı kılar. En küçük çalışan programla başlayalım:

```
1 #include <iostream>    // std::cout için standart giris/cikis akisi
2
3 // main: her programın baslangic noktasidir; isletim sistemi burayi cagirir
4 int main() {
5     std::cout << "Merhaba, Dünya!" << "\n";    // ekrana yaz
6     return 0;    // 0 = basarili cikis kodu
7 }
```

```
1 # Derleme ve calistirma (GCC)
2 g++ -std=c++17 -Wall -Wextra merhaba.cpp -o merhaba
3 ./merhaba
4 # Cikti: Merhaba, Dünya!
```

Bilgi

Derleme dört aşamadan geçer: **ön işleme** (#include, makrolar), **derleme** (kaynak → assembly), **çevirme** (assembly → nesne dosyası .o) ve **bağlama** (linking: nesne dosyaları + kütüphaneler → çalıştırılabilir). -Wall -Wextra bayraklarını her zaman açın; potansiyel hataları derleme aşamasında yakalar.

1.1 Başlık Dosyaları ve Ad Alanları (Namespaces)

Kod, *bildirimlerin* bulunduğu başlık dosyaları (.hpp) ve *tanımların* bulunduğu kaynak dosyalar (.cpp) olarak örgütlenir. namespace, isim çakışmalarını önlemek için sembollerini mantıksal gruplara ayırır.

```
1 #include <iostream>
2
3 // Kendi ad alanımız -- isim cakismalarini onler
4 namespace math {
5     const double PI = 3.14159265358979;
6     double square(double x) { return x * x; }
7 }
```

```

8     namespace geometry {           // ic ice ad alanı
9         double circle_area(double r) { return PI * square(r); }
10    }
11
12
13    int main() {
14        // Tam nitelenmiş isim (fully qualified)
15        std::cout << math::geometry::circle_area(2.0) << "\n";
16
17        // 'using' ile bir ismi kapsama getir
18        using math::square;
19        std::cout << square(5) << "\n";           // 25
20
21        // DİKKAT: 'using namespace std;' başlık dosyalarında ASLA kullanılmaz
22        // (tüm std'yi getirir, çakışmalara yol açar). .cpp içinde olc
23
24        return 0;
25    }

```

Dikkat

Başlık dosyalarında (.hpp) asla `using namespace std;` yazmayın. Bu satır, o başlığı dahil eden *her* dosyaya tüm `std` ad alanını enjekte eder ve gizli isim çakışmalarına yol açar. Kaynak dosyalarda (.cpp) ölçülü kullanılabilir; en güvenlisi her zaman `std::` önekini açıkça yazmaktır.

2. Temel Tipler, Değişkenler ve auto

C++ *statik tipli* bir dildir: her değişkenin tipi derleme zamanında bilinir ve sabittir. Temel (yerleşik) tipler bellekte belirli boyutlar kaplar ve belirli değer aralıklarına sahiptir.

Tip	Tipik Boyut	Açıklama
bool	1 bayt	true / false
char	1 bayt	Karakter / küçük tam sayı
int	4 bayt	Tam sayı ($\approx \pm 2.1$ milyar)
long long	8 bayt	Büyük tam sayı
float	4 bayt	Tek hassasiyetli ondalık
double	8 bayt	Çift hassasiyetli ondalık
std::size_t	8 bayt	İşaretsiz boyut/indeks tipi

```

1  #include <iostream>
2  #include <string>
3  #include <cstdint>
4
5  int main() {
6      // Modern C++: kume parantezli baslatma (uniform initialization)
7      int    age{25};           // brace-init: daraltmaya (narrowing) izin vermez
8      double pi{3.14159};
9      bool   active{true};
10     char    letter{'A'};
11     std::string name{"Ada"};

```

```

12 // int hata{3.14};           // HATA: double -> int daraltma engellenir (guvenli!)
13
14
15 // 'auto': tipi derleyici initializer'dan CIKARIR
16 auto number = 42;           // int
17 auto ratio = 3.5;           // double
18 auto text = std::string{"otomatik"}; // std::string
19 auto& ref = age;             // int& (referans icin & yaz)
20
21 // Tamsayi literalleri ve ayirici
22 auto big = 1'000'000;        // ' okunabilirlik ayiricisi (C++14)
23 std::uint64_t mask = 0xFF00; // onaltilik (hex)
24
25 std::cout << name << ", " << age << " (auto sayi=" << number << ")\n";
26
27 // const ve constexpr
28 const int CONSTANT = 100;    // calisma zamani sabiti
29 constexpr int COMPILE_CONSTANT = 50; // derleme zamani sabiti
30 std::cout << CONSTANT + COMPILE_CONSTANT << "\n";
31 return 0;
32 }

```

İpucu

auto, kodu kısaltır ve tip tekrarını önler; özellikle uzun iteratör tiplerinde (std::vector<int>::iterator yerine auto) çok değerlidir. Ancak niyeti belirsizleştirdiği yerlerde açık tip yazmak daha okunur olabilir. Kume parantezli başlatma ({}), ise “daraltma” (narrowing) dönüşümlerini derleme zamanında yakaladığı için en güvenli başlatma biçimidir.

3. Kontrol Akışı

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 int main() {
6     int score = 75;
7
8     // if / else if / else
9     if (score >= 90) std::cout << "AA\n";
10    else if (score >= 70) std::cout << "BB\n";
11    else std::cout << "Kaldi\n";
12
13    // switch: tam sayi/enum degerlerine dallan
14    int day = 3;
15    switch (day) {
16        case 1: std::cout << "Pazartesi\n"; break;
17        case 3: std::cout << "Carsamba\n"; break;
18        default: std::cout << "Diger\n"; break;
19    }
20
21    // Klasik for
22    for (int i = 0; i < 5; ++i) std::cout << i << " ";
23    std::cout << "\n";
24

```

```

25 // while ve do-while
26 int n = 3;
27 while (n > 0) { std::cout << n << " "; --n; }
28 std::cout << "\n";
29
30 // RANGE-BASED FOR (C++11): en cok tercih edilen dongu bicimi
31 std::vector<std::string> colors{"kirmizi", "yesil", "mavi"};
32 for (const auto& color : colors) // kopyalamadan gez (const ref)
33     std::cout << color << " ";
34 std::cout << "\n";
35
36 // Indeksle gezmek gerekiyorsa yapisal baglama ile (C++17)
37 for (std::size_t i = 0; i < colors.size(); ++i)
38     std::cout << i << ":" << colors[i] << " ";
39 std::cout << "\n";
40 return 0;
41 }

```

Bilgi

Range-based for (`for (auto& x : kap)`) modern C++'ın tercih edilen döngüsüdür: indeks hatası (off-by-one) riskini ortadan kaldırır ve niyeti nettir. Elemanları değiştirmeyecekseniz `const auto&`, değiştirecekseniz `auto&`, ucuz kopyalanabilir küçük tipler için `auto` kullanın.

4. Fonksiyonlar

Fonksiyonlar, kodu yeniden kullanılabilir birimlere böler. C++'ta parametreler değere, referansa veya sabit referansa göre geçirilebilir — bu seçim hem performansı hem anlamı belirler.

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4
5 // 1) DEGERE gore: kopya alir; kucuk tipler icin uygun
6 int square(int x) { return x * x; }
7
8 // 2) REFERANSA gore: cagiranin degiskenini DEGISTIRIR
9 void double_value(int& x) { x *= 2; }
10
11 // 3) SABIT REFERANSA gore: buyuk nesneyi KOPYALAMADAN, degistirmeden gecir
12 double average(const std::vector<double>& values) {
13     double sum = 0;
14     for (double v : values) sum += v;
15     return values.empty() ? 0.0 : sum / values.size();
16 }
17
18 // 4) VARSAYILAN argumanlar
19 void greet(const std::string& name, const std::string& title = "Sn.") {
20     std::cout << title << " " << name << ", merhaba!\n";
21 }
22
23 // 5) ASIRI YUKLEME (overloading): ayni isim, farkli parametre listeleri
24 int add(int a, int b) { return a + b; }
25 double add(double a, double b) { return a + b; }
26
27 // 6) Sondaki donus tipi (trailing return type) -- karmasik tiplerde

```



```
28 auto multiply(int a, int b) -> int { return a * b; }
29
30 int main() {
31     std::cout << square(6) << "\n";           // 36
32
33     int value = 10;
34     double_value(value);                       // referans -> deger degisir
35     std::cout << value << "\n";               // 20
36
37     std::cout << average({1.0, 2.0, 3.0, 4.0}) << "\n"; // 2.5
38
39     greet("Ada");                             // varsayilan unvan
40     greet("Turing", "Prof.");
41
42     std::cout << add(2, 3) << " " << add(2.5, 3.5) << "\n"; // 5 6
43     std::cout << multiply(4, 5) << "\n";       // 20
44     return 0;
45 }
```

İpucu

Parametre geçirme kuralı: Küçük, ucuz kopyalanan tipler (int, double, işaretçi) için *değere* geçirin. Büyük nesneleri (std::string, std::vector, sınıflar) yalnızca okuyacaksanız const T& ile geçirin — kopya maliyetinden kaçınır. Fonksiyonun çağırının nesnesini *değiştirmesi* gerekiyorsa T& (referans) kullanın.

KISIM 2

Nesne Yönelimli Programlama

5. Sınıflar ve Nesneler

Sınıf (class), veriyi (üye değişkenler) ve o veri üzerinde çalışan davranışı (üye fonksiyonlar) tek bir birimde toplayan kullanıcı tanımlı bir tiptir. Bu birleştirmeye **kapsülleme** (encapsulation) denir. Erişim belirleyiciler (private, public, protected) verinin dışarıdan nasıl görüneceğini kontrol eder.

```

1  #include <iostream>
2  #include <string>
3  #include <stdexcept>
4
5  class BankAccount {
6  private:                                // disaridan ERISILEMEZ (kapsulleme)
7      std::string owner_;
8      double balance_;
9
10 public:                                // genel arayuz (interface)
11     // Yapici (constructor): nesne olustugunda uyeleri baslatir
12     BankAccount(std::string owner, double initial = 0.0)
13         : owner_(std::move(owner)), balance_(initial) {
14         if (initial < 0) throw std::invalid_argument("Negatif bakiye");
15     }
16
17     // Uye fonksiyonlar (metotlar) -- veriyi kontrollu degistirir
18     void deposit(double amount) {
19         if (amount <= 0) throw std::invalid_argument("Gecersiz miktar");
20         balance_ += amount;
21     }
22
23     void withdraw(double amount) {
24         if (amount > balance_) throw std::runtime_error("Yetersiz bakiye");
25         balance_ -= amount;
26     }
27
28     // const uye fonksiyon: nesneyi DEGISTIRMEZ (yalnizca okur)
29     double balance() const { return balance_; }
30     const std::string& owner() const { return owner_; }
31 };
32
33 int main() {
34     BankAccount account("Ada Lovelace", 1000);
35     account.deposit(500);

```

```

36     account.withdraw(200);
37     std::cout << account.owner() << " bakiye: " << account.balance() << "\n"; // 1300
38
39     try {
40         account.withdraw(999999);           // yetersiz bakiye -> istisna
41     } catch (const std::exception& e) {
42         std::cout << "Hata: " << e.what() << "\n";
43     }
44     return 0;
45 }

```

Bilgi

Kapsülleme ilkesi: Üye değişkenleri private yapın ve dışarıya yalnızca kontrollü bir public arayüz sunun. Böylece sınıfın iç durumu her zaman tutarlı (invariant) kalır — yukarıda bakiye asla negatif olamaz, çünkü tek değiştirme yolu yatır/cek metotlarından geçer ve onlar doğrulama yapar. struct ile class teknik olarak yalnızca varsayılan erişim belirleyicisinde farklıdır (struct: public, class: private).

6. Kalıtım (Inheritance)

Kalıtım, mevcut bir sınıftan (*taban*, base) yeni bir sınıf (*türetilmiş*, derived) oluşturmayı sağlar. Türetilmiş sınıf, tabanın üyelerini devralır ve genişletir. Bu, “bir-türüdür” (is-a) ilişkisini modeller: bir Kopek, bir Hayvan’dır.

```

1  #include <iostream>
2  #include <string>
3
4  // TABAN sınıf
5  class Animal {
6  protected:                               // turetilmis siniflar erisebilir, disari erisemez
7      std::string name_;
8  public:
9      Animal(std::string name) : name_(std::move(name)) {}
10     void breathe() const { std::cout << name_ << " nefes aliyor\n"; }
11     const std::string& name() const { return name_; }
12 };
13
14 // TURETILMIS sınıf: Hayvan'dan public kalitim
15 class Dog : public Animal {
16     std::string breed_;
17 public:
18     // Taban yapicisini uye baslatma listesinde CAGIR
19     Dog(std::string name, std::string breed)
20         : Animal(std::move(name)), breed_(std::move(breed)) {}
21
22     void bark() const { std::cout << name_ << " (" << breed_ << "): Hav!\n"; }
23 };
24
25 int main() {
26     Dog k("Karabas", "Kangal");
27     k.breathe();           // TABAN'dan devralindi
28     k.bark();              // KENDI metodu
29     std::cout << k.name() << "\n"; // taban'in public metodu
30     return 0;

```

31 }

Kalıtım türü	Etkisi
public	Taban'ın public'i public, protected'ı protected kalır. (En yaygın, "is-a".)
protected	Taban'ın public ve protected üyeleri protected olur.
private	Taban'ın tüm üyeleri private olur. ("ile-gerçeklenmiştir".)

7. Polimorfizm ve Sanal Fonksiyonlar

Polimorfizm ("çok biçimlilik"), bir taban sınıf işaretçisi/referansı üzerinden çağrılan bir fonksiyonun, *çalışma zamanında* gerçek nesnenin tipine göre doğru sürümünü çalıştırmasıdır. Bu, virtual anahtar kelimesiyle sağlanır ve nesne yönelimli tasarımın en güçlü aracıdır.

```

1  #include <iostream>
2  #include <vector>
3  #include <memory>
4
5  // Soyut taban sınıf (abstract base class / arayuz)
6  class Shape {
7  public:
8      // Saf sanal fonksiyon (= 0): goudesi yok, alt sınıflar MECBUREN yazar
9      virtual double area() const = 0;
10     virtual void draw() const = 0;
11
12     // SANAL YIKICI: taban isaretcisiyle silmede kritik!
13     virtual ~Shape() = default;
14 };
15
16 class Circle : public Shape {
17     double radius_;
18 public:
19     Circle(double r) : radius_(r) {}
20     double area() const override { return 3.14159 * radius_ * radius_; }
21     void draw() const override { std::cout << "Daire (r=" << radius_ << ")\n"; }
22 };
23
24 class Rectangle : public Shape {
25     double width_, height_;
26 public:
27     Rectangle(double e, double b) : width_(e), height_(b) {}
28     double area() const override { return width_ * height_; }
29     void draw() const override { std::cout << "Dikdortgen " << width_ << "x" << height_ << "\n"; }
30 };
31
32 int main() {
33     // Taban isaretcileriyle farkli turleri TEK kaptaki tut (polimorfizm)
34     std::vector<std::unique_ptr<Shape>> shapes;
35     shapes.push_back(std::make_unique<Circle>(2.0));
36     shapes.push_back(std::make_unique<Rectangle>(3.0, 4.0));

```

```
37
38     double total = 0;
39     for (const auto& s : shapes) {
40         s->draw();           // dogru ciz() CALISMA ZAMANINDA secilir
41         total += s->area();   // dogru alan() cagrilir
42     }
43     std::cout << "Toplam alan: " << total << "\n";
44     return 0;
45 } // unique_ptr'lar silinirken SANAL yıkici sayesinde dogru yıkici cagrilir
```

Dikkat

Sanal yıkıcı kritiktir: Bir sınıf polimorfik olarak kullanılacaksa (taban işaretçisiyle türetilmiş nesne tutulup silinecekse), tabanın yıkıcısı virtual olmalıdır. Aksi halde delete taban_ptr yalnızca tabanın yıkıcısını çağırır, türetilmişinkini *atlar* — kaynak sızıntısı ve tanımsız davranış. Kural: “Sanal fonksiyonu olan her sınıfın sanal yıkıcısı olmalıdır.”

Bilgi

override anahtar kelimesi zorunlu değildir ama her zaman kullanın: derleyici, gerçekten bir taban sanal fonksiyonunu geçersiz kılıp kılmadığınızı kontrol eder. İmzada küçük bir hata (örneğin const unutmak) yaparsanız, override olmadan sessizce *yeni* bir fonksiyon tanımlarsınız; override ile derleme hatası alır ve hatayı anında görürsünüz. final ise bir sınıfın/fonksiyonun daha fazla türetilmesini/geçersiz kılınmasını engeller.

KISIM 3

Bellek Modeli ve İşaretçiler

8. Programın Bellek Düzeni

Bir C++ programı çalışırken belleği birkaç mantıksal bölgeye ayırır. Bu bölgeleri anlamak, kaynak yönetiminin temelidir; çünkü bir nesnenin *nerede* yaşadığı, ne kadar yaşadığını ve onu kimin temizlemesi gerektiğini belirler.

Bölge	Açıklama
Kod (text)	Makine talimatları. Salt okunur, sabit boyut.
Statik/Global	Global ve static değişkenler. Program boyunca yaşar.
Yığın (stack)	Yerel değişkenler, fonksiyon çağrı çerçeveleri. Otomatik ömür: kapsam bitince <i>otomatik</i> yok edilir. Hızlı ama sınırlı.
Öbek (heap)	new/malloc ile ayrılan bellek. Ömrü programcı yönetir. Esnek ama pahalı ve sızıntıya açık.

```

1  #include <iostream>
2
3  int global_value = 10;           // statik/global bolge
4
5  void func() {
6      int local = 5;               // YIĞIN (stack): kapsam bitince otomatik silinir
7      static int calls = 0;        // STATİK: çağrılar arası değerini korur
8      int* heap_val = new int(42); // ÖBEK (heap): ELLE silinmeli!
9
10     ++calls;
11     std::cout << "yerel=" << local << " çağrı=" << calls
12               << " obek=" << *heap_val << "\n";
13
14     delete heap_val;              // AKSI HALDE BELLEK SIZINTISI
15 } // 'yerel' burada otomatik yok olur; 'obekten' isaretçisi de,
16   // ama isaret ettiği obek belleği delete edilmezse sızardı.
17
18 int main() {
19     func();
20     func();

```

```

21     return 0;
22 }

```

Bilgi

Otomatik ömür (automatic storage) C++'ın en güçlü özelliğidir: yığındaki nesneler, kapsam (scope, yani { }) bittiğinde derleyici tarafından *garantili* olarak yok edilir; siz hiçbir şey yapmazsınız. Bu garanti, ileride göreceğimiz RAIİ deseninin temelidir. Öbekteki nesnelerde ise böyle bir garanti yoktur; onları siz temizlemekle yükümlüsünüz.

9. new ve delete: Öbek Belleği Yönetimi

`new` operatörü öbekte bellek ayırır *ve* nesnenin yapıcısını çağırır. `delete` ise yıkıcıyı çağırır *ve* belleği geri verir. Bu ikisi C'deki `malloc/free`'den farklıdır: onlar yapıcı/yıkıcı çalıştırmaz, yalnızca ham bellek yönetir.

```

1  #include <iostream>
2
3  struct Resource {
4      int id;
5      Resource(int i) : id(i) { std::cout << "Kaynak " << id << " olustu\n"; }
6      ~Resource()           { std::cout << "Kaynak " << id << " yok oldu\n"; }
7  };
8
9  int main() {
10     // Tekil nesne
11     Resource* k = new Resource(1); // bellek ayIr + yapIcI cagIr
12     std::cout << "id = " << k->id << "\n";
13     delete k; // yIkIcI cagIr + bellek geri ver
14
15     // Dizi: new[] ve delete[] BIRLIKTE kullanIlmalI
16     Resource* arr = new Resource[3]{ {10}, {20}, {30} };
17     std::cout << "dizi[1].id = " << arr[1].id << "\n";
18     delete[] arr; // MUTLAKE delete[] -- 'delete' YANLIS olur
19
20     return 0;
21 }

```

Dikkat

`new[]` ile ayrılan bellek `delete[]` ile, `new` ile ayrılan `delete` ile serbest bırakılmalıdır. `new[]` belleğini düz `delete` ile silmek **tanımsız davranıştır** (undefined behavior); genellikle yalnızca ilk elemanın yıkıcısı çağrılır ve bellek bozulur. Modern C++'ta bu hatalardan kaçınmak için *ham* `new/delete` yerine akıllı işaretçileri ve konteynerleri (`std::vector`) tercih edin.

9.1 Bellek Sızıntıları ve İkili Silme

En sık yapılan iki hata: belleği hiç silmemek (sızıntı) veya aynı belleği iki kez silmek (double free).

```

1  void leak() {
2      int* p = new int(5);
3      // ... delete unutuldu -> SIZINTI: bu bellek program bitene kadar kayIp
4  }
5

```

```

6 void double_delete() {
7     int* p = new int(5);
8     delete p;
9     delete p;      // TANIMSIZ DAVRANIS: ayni bellegi iki kez silmek
10    // Cozum: sildikten sonra p = nullptr; yapmak
11    // (nullptr'I delete etmek guvenlidir, hicbir sey yapmaz)
12 }
13
14 void dangling() {
15     int* p = new int(5);
16     delete p;
17     std::cout << *p;    // SARKAN ISARETCI: silinen bellege erisim -> UB
18     p = nullptr;        // dogru aliskanlik
19 }

```

İpucu

Bu üç sorunun (sızıntı, ikili silme, sarkan işaretçi) *kalıcı* çözümü, ham `new/delete`'i elle yönetmeyi bırakıp sahipliği akıllı işaretçilere devretmektir. Kısım 2'de bunu göreceğiz. Kural: “Modern C++ kodunda çıplak `new` ve `delete` neredeyse hiç görünmemelidir.”

10. İşaretçiler ve Referanslar

İşaretçi (pointer), başka bir nesnenin *bellek adresini* tutan bir değişkendir. Referans (reference) ise mevcut bir nesneye verilen *takma addır* (alias). İkisi de dolaylı erişim sağlar ama davranışları önemli ölçüde farklıdır.

```

1 #include <iostream>
2
3 int main() {
4     int x = 10;
5
6     // ISARETCI: x'in adresini tutar; yeniden atanabilir, null olabilir
7     int* p = &x;      // p, x'i gösterir
8     *p = 20;          // dereference: x'in degerini degistirir
9     std::cout << "x = " << x << "\n";    // 20
10
11     int y = 99;
12     p = &y;           // isaretcisi baskasini gosterecek sekilde DEGISTIRILEBILIR
13     std::cout << "*p = " << *p << "\n"; // 99
14
15     // REFERANS: x'e takma ad; olusturuldugu anda baglanir, degistirilemez
16     int& r = x;        // r, x'in ta kendisidir
17     r = 30;            // x'i degistirir (yeni degisken DEGIL)
18     std::cout << "x = " << x << "\n";    // 30
19     // int& r2;        // HATA: referans mutlaka baslangicta baglanmalI
20     // r = y;          // bu y'yi r'ye atamaz; x = y anlamina gelir!
21
22     return 0;
23 }

```


Özellik	İşaretçi (T*)	Referans (T&)
Null olabilir mi?	Evet (nullptr)	Hayır
Yeniden bağlanır mı?	Evet, başkasını gösterebilir	Hayır, sabit bağ
Başlatma zorunlu mu?	Hayır (ama tehlikeli)	Evet
Aritmetik yapılır mı?	Evet (p+1)	Hayır
Adres alınır mı?	&p işaretçinin adresi	&r nesnenin adresi
Kullanım	Sahiplik, dizi, opsiyonel	Fonksiyon parametresi, takma ad

Bilgi

Ne zaman hangisi? Bir fonksiyona “değiştirilebilir ama her zaman geçerli” bir nesne geçiriyorsanız **referans** (T&) kullanın. “Opsiyonel” (olmayabilir) veya “yeniden yönlendirilebilir” bir bağlantı gerekiyorsa **işaretçi** kullanın. Büyük nesneleri kopyalamadan salt okunur geçirmek için `const T&` idealdir.

10.1 const ve İşaretçiler

`const` anahtar kelimesinin işaretçilerle konumu, neyin sabit olduğunu değiştirir. Bu, C++’ta sık kafa karıştıran ama kritik bir konudur. Okumak için işaretçiye *sağdan sola* okuyun.

```

1  int value = 10, other = 20;
2
3  // 1) Gosterilen SABIT, isaretci degisebilir
4  const int* p1 = &value;      // "pointer to const int"
5  // *p1 = 5;                  // HATA: gosterilen degeri degistiremez
6  p1 = &other;                 // OK: isaretci baskasini gosterebilir
7
8  // 2) Isaretci SABIT, gosterilen degisebilir
9  int* const p2 = &value;      // "const pointer to int"
10 *p2 = 5;                     // OK: gosterilen degeri degistirir
11 // p2 = &other;              // HATA: isaretci yeniden atanamaz
12
13 // 3) Her ikisi de SABIT
14 const int* const p3 = &value;
15 // *p3 = 5;                  // HATA
16 // p3 = &baska;              // HATA
17
18 // Okuma kurali (sağdan sola):
19 //  const int* const p3
20 //  p3, const bir (isaretci) -> const int'e

```

İpucu

Pratik kural: `const`’ı işaretçi yıldızının *soluna* yazarsanız *gösterilen* sabittir (`const int* p`); *sağına* yazarsanız *işaretçinin kendisi* sabittir (`int* const p`). Salt okunur veri geçirmek için en sık kullanılan biçim birincisidir: `const T*` veya `const T&`.

10.2 Boş, Sarkan ve Yaban İşaretçiler

```
1 int* wild; // YABAN (wild): baslatilmadi, cop adres -> UB
2 int* null_ptr = nullptr; // BOS (null): guvenli, "hicbir seyi gostermiyor"
3
4 if (null_ptr == nullptr) // guvenli kontrol
5     std::cout << "bos isaretci\n";
6
7 int* dangling = new int(5);
8 delete dangling; // bellek geri verildi
9 // sarkan artik SARKAN (dangling): gecersiz bellegi gosteriyor
10 dangling = nullptr; // dogru alIskanlik: sildikten sonra sIfIrla
11
12 // nullptr her zaman C++11+ ile kullanilmali; eski 'NULL' veya '0' DEGIL
13 void f(int);
14 void f(char*);
15 // f(NULL); // belirsiz olabilir (NULL genelde 0'dir -> f(int))
16 // f(nullptr); // net: f(char*) cagrIlIr
```

Dikkat

nullptr, C++11 ile gelen tip güvenli boş işaretçi sabitidir. Eski NULL makrosu genellikle tam sayı 0 olarak tanımlıdır ve aşırı yüklenmiş fonksiyonlarda yanlış olamı seçebilir. Modern kodda her zaman nullptr kullanın.

KISIM 4

RAII ve Akıllı İşaretçiler

11. RAII: Kaynak Edinimi Başlatmadır

RAII (Resource Acquisition Is Initialization), C++'ın en önemli deyimidir. Fikir basittir: bir *kaynağı* (bellek, dosya, kilit, soket) bir *nesnenin* yapıcısında edin, yıkıcısında serbest bırak. Nesne yığında yaşadığından, kapsam bittiğinde yıkıcı *otomatik* ve *garantili* çalışır — istisna atılsa bile. Böylece kaynak asla sızmaz.

```

1  #include <iostream>
2
3  // RAII sarmalayıcı: bir dosya tanıtıcısı (handle) yönetir
4  class FileHolder {
5      std::FILE* file_;
6  public:
7      explicit FileHolder(const char* name, const char* mode)
8          : file_(std::fopen(name, mode)) {
9          if (!file_) throw std::runtime_error("Dosya acılamadı");
10         std::cout << "Dosya açıldı\n";
11     }
12     ~FileHolder() { // YIKICI: kaynağı serbest bırak
13         if (file_) { std::fclose(file_); std::cout << "Dosya kapandı\n"; }
14     }
15     std::FILE* get() const { return file_; }
16
17     // Kopyalamayı engelle (kaynak tekil sahiplikte -- Kisim 3'te açıklanacak)
18     FileHolder(const FileHolder&) = delete;
19     FileHolder& operator=(const FileHolder&) = delete;
20 };
21
22 void write() {
23     FileHolder f("/tmp/test.txt", "w");
24     std::fputs("Merhaba RAII\n", f.get());
25     // throw std::runtime_error("hata!"); // istisna olsa BİLE dosya kapanır
26 } // kapsam bitti -> f'nin yıkıcısı otomatik çağılır -> dosya kapanır
27
28 int main() {
29     write();
30     std::cout << "Bitti\n";
31     return 0;
32 }

```

Bilgi

RAII'nin gücü **istisna güvenliğidir**: fonksiyonun ortasında bir istisna atılsa bile, yığın çözülürken (stack unwinding) tüm yerel nesnelerin yıkıcıları çalışır. Böylece dosya kapatma, kilit bırakma, bellek serbest bırakma gibi temizlik işlemleri *unutulamaz*. `std::vector`, `std::string`, `std::lock_guard` ve akıllı işaretçiler — hepsi RAII uygular.

12. std::unique_ptr: Tekil Sahiplik

`std::unique_ptr`, bir öbek nesnesinin *tek* sahibidir. Kopyalanamaz (iki sahip olamaz) ama *taşınabilir* (sahiplik devri). Kapsam bittiğinde işaret ettiği nesneyi otomatik siler. Ham işaretçiyle aynı boyuttadır — yani sıfır ek yük (zero overhead).

```

1  #include <iostream>
2  #include <memory>
3
4  struct Object {
5      int id;
6      Object(int i) : id(i) { std::cout << "Nesne " << id << " olustu\n"; }
7      ~Object()           { std::cout << "Nesne " << id << " silindi\n"; }
8      void greet() const { std::cout << "Ben " << id << "\n"; }
9  };
10
11 std::unique_ptr<Object> produce(int id) {
12     // make_unique: istisna guvenli, tercih edilen olusturma yolu
13     return std::make_unique<Object>(id);
14 } // sahiplik cagIrana TASINIR (move) -- kopyalanmaz
15
16 int main() {
17     std::unique_ptr<Object> a = std::make_unique<Object>(1);
18     a->greet();                // -> operatoru ham isaretci gibi calisir
19
20     // KOPYALAMA YASAK -- derleme hatasi:
21     // std::unique_ptr<Object> b = a; // HATA: unique_ptr kopyalanamaz
22
23     // TASIMA serbest -- sahiplik a'dan b'ye gecer
24     std::unique_ptr<Object> b = std::move(a);
25     if (!a) std::cout << "a artik bos (nullptr)\n";
26     b->greet();
27
28     // Fonksiyondan donen sahipligi al
29     std::unique_ptr<Object> c = produce(2);
30
31     // release / reset
32     Object* raw = c.release();    // sahipligi BIRAK, ham isaretci al (silme SANA kalIr)
33     delete raw;                  // artik elle silmelisin
34
35     b.reset();                   // b'nin nesnesini simdi sil (Nesne 1 silinir)
36     std::cout << "main sonu\n";
37     return 0;
38 } // program sonu: kalan tum unique_ptr'lar nesnelerini otomatik siler

```

İpucu

Her zaman `std::make_unique<T>(...)` kullanın, `unique_ptr<T>(new T(...))` değil. `make_unique` daha kısadır, T'yi tekrar yazmaz ve istisna güvenlidir. `unique_ptr`, dizi için de çalışır: `std::unique_ptr<int[]> d = std::make_unique<int[]>(100);` otomatik olarak `delete[]` kullanır.

12.1 Özel Silici (Custom Deleter)

`unique_ptr`, belleği nasıl serbest bırakacağını özelleştirmenize izin verir. Bu, `delete` dışında bir temizlik gerektiren kaynaklar (C API tanıtıcıları, `fclose`, `free`) için idealdir.

```
1 #include <cstdio>
2 #include <memory>
3
4 int main() {
5     // FILE* için fclose ile temizleyen unique_ptr
6     auto deleter = [](std::FILE* f){ if (f) std::fclose(f); };
7     std::unique_ptr<std::FILE, decltype(deleter)> file(
8         std::fopen("/tmp/veri.txt", "w"), deleter);
9
10    if (file) std::fputs("ozel silici ile\n", file.get());
11    return 0;
12 } // dosya kapsam dışı -> lambda silici çağrılır -> fclose
```

13. std::shared_ptr ve std::weak_ptr: Paylaşımlı Sahiplik

`std::shared_ptr`, bir nesnenin *birden fazla* sahibi olmasına izin verir. Bir *referans sayacı* tutar: her kopya sayacı artırır, her yok olma azaltır. Sayacı sıfıra düştüğünde nesne silinir. Bu esneklik bir maliyetle gelir: sayacı için ek bir kontrol bloğu ve atomik güncellemeler.

```
1 #include <iostream>
2 #include <memory>
3
4 struct Node {
5     int value;
6     Node(int v) : value(v) { std::cout << "Dugum " << v << " olustu\n"; }
7     ~Node() { std::cout << "Dugum " << value << " silindi\n"; }
8 };
9
10 int main() {
11     std::shared_ptr<Node> a = std::make_shared<Node>(1);
12     std::cout << "Sayac: " << a.use_count() << "\n"; // 1
13
14     {
15         std::shared_ptr<Node> b = a; // KOPYA: sayac artar
16         std::cout << "Sayac: " << a.use_count() << "\n"; // 2
17         b->value = 100; // aynı nesne (a ile paylaşımlı)
18         std::cout << "a->deger = " << a->value << "\n"; // 100
19     } // b yok oldu -> sayac azalır
20
21     std::cout << "Sayac: " << a.use_count() << "\n"; // 1
22     return 0;
23 } // a yok oldu -> sayac 0 -> "Dugum 1 silindi"
```

Dikkat

`shared_ptr` her nesne için ayrı bir kontrol bloğu tahsis eder ve sayaç güncellemeleri iş parçacığı güvenliği için atomiktir — bu, `unique_ptr`'a göre gözle görülür bir ek yükür. **Varsayılan tercihiniz her zaman `unique_ptr` olmalıdır**; yalnızca sahipliğin gerçekten *paylaşılması* gerektiğinde `shared_ptr`'a geçin. `make_shared`, nesneyi ve kontrol bloğunu tek tahsiste birleştirerek performansı iyileştirir.

13.1 Döngüsel Referans Sorunu ve `weak_ptr`

`shared_ptr`'ın en büyük tuzağı **döngüsel referanstır**: iki nesne birbirini `shared_ptr` ile tutarsa, sayaçları asla sıfıra düşmez ve ikisi de asla silinmez — bir bellek sızıntısı. Çözüm, döngüyü kıran taraflardan birinde `std::weak_ptr` kullanmaktır. `weak_ptr`, nesneyi *sahiplenmeden* gözlemler; sayacı etkilemez.

```

1 #include <iostream>
2 #include <memory>
3
4 struct Child;
5 struct Parent {
6     std::shared_ptr<Child> child;
7     ~Parent() { std::cout << "Ebeveyn silindi\n"; }
8 };
9 struct Child {
10     // DİKKAT: ebeveyni shared_ptr ile tutsaydı DONGU olurdu.
11     // weak_ptr ile sahiplenmeden gözlemliyoruz:
12     std::weak_ptr<Parent> parent;
13     ~Child() { std::cout << "Cocuk silindi\n"; }
14 };
15
16 int main() {
17     auto e = std::make_shared<Parent>();
18     auto c = std::make_shared<Child>();
19
20     e->child = c;           // Ebeveyn -> Cocuk (shared, sahiplik)
21     c->parent = e;          // Cocuk -> Ebeveyn (weak, gözlem) -> DONGU YOK
22
23     // weak_ptr'I kullanmak için önce lock() ile shared_ptr'a çevir
24     if (auto locked = c->parent.lock()) {
25         std::cout << "Ebeveyn hala yaşıyor, sayac: "
26                 << locked.use_count() << "\n";
27     }
28     return 0;
29 } // Her iki nesne de doğru şekilde silinir (dongu kırıldı)

```

Bilgi

`weak_ptr`, “sahiplenmeyen gözlemci” rolündedir. Nesneye erişmek için önce `lock()` çağırıp geçici bir `shared_ptr` elde edersiniz; nesne çoktan silinmişse `lock()` boş bir `shared_ptr` döndürür ve siz güvenle kontrol edebilirsiniz. Ebeveyn-çocuk, gözlemci (observer), önbellek gibi ilişkilerde döngüyü kırmak için idealdir.

İşaretçi	Ne zaman kullanılır
unique_ptr	Varsayılan. Tekil sahiplik. Sıfır ek yük.
shared_ptr	Sahiplik gerçekten paylaşılmalıysa. Atomik sayaç, ek yük.
weak_ptr	Sahiplenmeden gözlem; shared_ptr döngülerini kırmak.
Ham T*	Sahiplenmeyen, salt gözlem/parametre. <i>Asla delete etmeyin.</i>

KISIM 5

Özel Üye Fonksiyonlar — Beşin Kuralı

14. Değer Kategorileri: lvalue ve rvalue

Beşin Kuralını anlamak için önce *değer kategorilerini* anlamak gerekir. Basitçe: **lvalue**, bir adı/adresi olan, kalıcı bir nesnedir (soluna atama yapılabilir). **rvalue**, geçici, isimsiz, birazdan yok olacak bir değerdir (bir ifadenin sonucu, geçici nesne).

```

1  #include <string>
2
3  std::string produce() { return "gecici"; }
4
5  int main() {
6      std::string a = "merhaba";    // a bir LVALUE (isimli, kalıcı)
7      std::string b = a;            // a lvalue -> KOPYA yapılır
8
9      std::string c = produce();    // uret()'in dondurdugu gecici bir RVALUE
10                                     // -> TASIMA (move) yapılabilir (ucuz)
11      std::string d = a + b;        // (a+b) gecici bir RVALUE
12
13      // std::move: bir lvalue'yu rvalue'ymus GIBI davranmaya "iter"
14      std::string e = std::move(a); // a'nın içeriği e'ye TASINIR
15      // artık a "geçerli ama belirsiz" durumda -- yeniden atamadan kullanmayın
16      return 0;
17  }
```

Bilgi

rvalue referansı (T&&), yalnızca rvalue'lara bağlanabilen özel bir referanstır ve C++11'in taşıma semantiğinin temelidir. Bir nesne “birazdan zaten yok olacaksa”, kaynaklarını *kopyalamak* yerine *çalmak* (taşımak) çok daha ucuzdur. İşte taşıma yapıcı ve taşıma atama operatörü tam olarak bunu yapar.

Dikkat

`std::move` aslında hiçbir şeyi *taşımaz*! Yalnızca bir statik dönüşümdür: lvalue'yu rvalue referansına çevirerek derleyiciye “bu nesnenin kaynaklarını çalabilirsin” der. Gerçek taşıma işini, çağrılan taşıma yapıcı/atama operatörü yapar. `std::move`'dan sonra kaynak nesne “geçerli ama belirsiz” durumdadır; yeniden atamadan içeriğine güvenmeyin.

15. Yıkıcı, Kopya ve Taşıma — Beş Fonksiyon Bir Arada

Kaynak (öbek belleği, dosya, soket...) yöneten her sınıfın, nesnenin tüm *yaşam döngüsünü* kontrol eden **beş özel üye fonksiyonu** vardır. Bunlar ayrı ayrı değil, bir *aile* olarak düşünülmelidir; çünkü hepsi aynı kaynağı doğru ele almak zorundadır. Derleyici bu beşini kendisi üretebilir, ancak sınıf elle bir kaynağı (new ile ayrılmış bellek) yönetiyorsa, üretilen varsayılanlar **yanlıştır**: işaretçiyi olduğu gibi kopyalarlar (sığ kopya) ve aynı bellek iki kez silinir. Bu yüzden beşini de bilinçli tanımlamak gerekir.

#	Fonksiyon	İmza (T için)	Ne zaman çağrılır
1	Yıkıcı	~T()	Nesne yok olduğunda
2	Kopya yapıcı	T(const T&)	Yeni nesne, mevcuttan <i>kopyayla</i> kurulur
3	Kopya atama	T& operator=(const T&)	Var olan nesneye lvalue atanır
4	Taşıma yapıcı	T(T&&) noexcept	Yeni nesne, rvalue'dan kurulur
5	Taşıma atama	T& operator=(T&&) noexcept	Var olan nesneye rvalue atanır

15.1 Beşi Bir Arada: Tam Çalışan Örnek

Aşağıdaki TextHolder sınıfı, öbekte bir karakter dizisi (char*) yönetir ve beş özel üye fonksiyonun *tümünü* tanımlar. Her fonksiyon hangi işlemin tetiklendiğini ekrana yazar; böylece main içindeki her satırın hangi fonksiyonu çağırdığını çıktıdan net görebilirsiniz.

```

1  #include <iostream>
2  #include <cstring>
3  #include <utility>
4
5  class TextHolder {
6      std::size_t size_;    // NOT: size_ önce bildirilir ki data_'dan ONCE başlasın
7      char* data_;         // sahip olunan öbek kaynağı
8  public:
9      // (0) Normal yapıcı -- kaynağı EDİNİR (RAII)
10     explicit TextHolder(const char* s = "")
11         : size_(std::strlen(s)), data_(new char[size_ + 1]) {
12         std::strcpy(data_, s);
13         std::cout << "[yapıcı]          \"\" << data_ << "\"\\n\"";
14     }
15
16     // (1) YIKICI -- kaynağı SERBEST BIRAKIR
17     ~TextHolder() {
18         std::cout << "[yıkıcı]          \"\" << (data_ ? data_ : "(bos)") << "\"\\n\"";
19         delete[] data_;
20     }
21
22     // (2) KOPYA YAPICI -- yeni, BAGIMSIZ kaynak ayırıp içerdiği kopyalar (derin kopya)
23     TextHolder(const TextHolder& other)

```

```

24     : size_(other.size_), data_(new char[other.size_ + 1]) {
25     std::strcpy(data_, other.data_);
26     std::cout << "[kopya yapici]   \"" << data_ << "\"\n";
27 }
28
29 // (3) KOPYA ATAMA -- var olan nesneye derin kopya atar
30 TextHolder& operator=(const TextHolder& other) {
31     std::cout << "[kopya atama]   \"" << other.data_ << "\"\n";
32     if (this != &other) {
33         char* fresh = new char[other.size_ + 1]; // kendine-atama koruması
34         std::strcpy(fresh, other.data_);         // 1) önce yeniyi ayır
35         delete[] data_;                          // (istisna güvenliği)
36         data_ = fresh;                           // 2) sonra eskiyi bırak
37         size_ = other.size_;                     // 3) yeni duruma geç
38     }
39     return *this;                               // zincirleme için (a = b = c)
40 }
41
42 // (4) TASIMA YAPICI -- kaynagi CALAR, orijinali gecerli-bos birakir (noexcept)
43 TextHolder(TextHolder&& other) noexcept
44     : size_(other.size_), data_(other.data_) { // isaretciyi DEVRAL
45     other.data_ = nullptr;                   // orijinali BOSALT
46     other.size_ = 0;
47     std::cout << "[tasima yapici]   \"" << data_ << "\"\n";
48 }
49
50 // (5) TASIMA ATAMA -- eski kaynagi birak, yeniyi cal (noexcept)
51 TextHolder& operator=(TextHolder&& other) noexcept {
52     std::cout << "[tasima atama]   \"" << (other.data_ ? other.data_ : "") << "\"\n";
53     if (this != &other) {
54         delete[] data_;                      // eski kaynagi birak
55         size_ = other.size_;
56         data_ = other.data_;                 // yeni kaynagi CAL
57         other.data_ = nullptr;               // orijinali bosalt
58         other.size_ = 0;
59     }
60     return *this;
61 }
62
63 const char* get() const { return data_ ? data_ : "(bos)"; }
64 };
65
66 TextHolder makeTemp() { return TextHolder("gecici"); } // gecici (rvalue) iletir
67
68 int main() {
69     TextHolder a("A"); // (0) yapici
70     TextHolder b = a;  // (2) kopya yapici -> yeni nesne, a'dan
71     TextHolder c("C"); // (0) yapici
72     c = a;             // (3) kopya atama -> c zaten VAR
73     TextHolder d = std::move(a); // (4) tasima yapici -> a'nin kaynagi calinir
74     TextHolder e("E"); // (0) yapici
75     e = makeTemp();    // (5) tasima atama -> sagda gecici (rvalue)
76
77     std::cout << "d = " << d.get() << ", a = " << a.get() << "\n";
78     return 0;
79 } // kapsam sonu: e, d, c, b, a için (1) yıkıcı -- TERS sırada

```

Program Çıktısı

```

[yapici]      "A"
[kopya yapici] "A"
[yapici]      "C"
[kopya atama]  "A"
[tasima yapici] "A"
[yapici]      "E"
[yapici]      "gecici"      (makeTemp icindeki gecici)
[tasima atama] "gecici"
[yikici]      "(bos)"      (gecici, tasindiktan sonra bos)
d = A, a = (bos)
[yikici]      "gecici"      (e)
[yikici]      "A"           (d)
[yikici]      "A"           (c)
[yikici]      "A"           (b)
[yikici]      "(bos)"      (a, kaynagi calinmisti)

```

15.2 1. Yıkıcı (Destructor)

Yıkıcı, nesne yok olduğunda *otomatik* çağrılır ve nesnenin sahip olduğu kaynağı serbest bırakır. $\tilde{}$ () biçiminde yazılır; parametre almaz, değer döndürmez, aşırı yüklenemez. Yukarıda `delete[] data_` ile öbek belleğini geri verir. Çıktıda görüldüğü gibi, yerel nesneler *kuruluşun tersi sırada* (e, d, c, b, a) yok edilir.

Dikkat

Bir sınıf polimorfik kullanılacaksa (taban işaretçisiyle türetilmiş nesne tutulup silinecekse), yıkıcısı *virtual* olmalıdır — `virtual ~Base() = default;`. Aksi halde `delete base_ptr` türetilmiş sınıfın yıkıcısını atlar ve kaynak sızar.

15.3 2. Kopya Yapıcı (Copy Constructor)

Kopya yapıcı, *yeni* bir nesneyi var olan başka bir nesnenin kopyası olarak *iklendirir* (`TextHolder b = a;`). Kritik nokta: **derin kopya** yapması gerekir — yeni, bağımsız bir öbek ayırıp içeriği oraya kopyalar. Böylece a ile b tamamen ayrı belleklere sahip olur.

Dikkat

Sığ kopya tuzağı: Kopya yapıcı yazmazsanız, derleyici işaretçiyi olduğu gibi kopyalayan bir varsayılan üretir — iki nesne *aynı* öbeği gösterir. Yıkıcıda önce biri, sonra diğeri *aynı* belleği siler: **ikili silme** (double free) ve çökme. Derin kopya tam olarak bunu önler.

Sığ vs Derin Kopya

```

SİĞ (yanlış):
a.data_ --.
> [ "A\0" ]      (TEK öbek, iki sahip!)
b.data_ --'

DERİN (doğru):
a.data_ --> [ "A\0" ]
b.data_ --> [ "A\0" ]      (iki AYRI öbek)

```

15.4 3. Kopya Atama Operatörü (Copy Assignment)

Kopya atama, *zaten var olan* bir nesneye başka bir nesnenin değerini *atar* (`c = a;`). Kopya yapıcıdan farkı: hedef nesnenin *eski bir kaynağı vardır* ve önce onu temizlemek gerekir. Doğru bir kopya atamada üç kritik nokta:

1. **Kendine atama koruması:** `if (this != &other) — x = x` durumunda önce kendi kaynağını silip sonra ondan kopyalamaya çalışmayı önler.
2. **İstisna güvenliği:** Önce yeni belleği ayırın, *sonra* eskiyi silin. Ayırma başarısız olursa (istisna), nesne eski geçerli durumunda kalır.
3. **Zincirleme için `*this` döndürün:** `a = b = c` ifadesinin çalışması için.

15.5 4. Taşıma Yapıcı (Move Constructor)

Kopyalama pahalıdır: yeni öbek ayırıp içeriği baytı baytına kopyalar. Ama kaynak nesne zaten *geçici* (rvalue) ise ve birazdan yok olacaksa, kaynağını *kopyalamak* yerine *çalmak* çok daha ucuzdur: sadece işaretçiyi devralır, orijinali `nullptr` yaparsınız. Bu, C++11'in taşıma semantiğidir ve modern C++ performansının temelidir. `TextHolder d = std::move(a);` satırı bunu tetikler — hiç bellek kopyalanmaz, yalnızca işaretçi `a`'dan `d`'ye geçer.

İpucu

Taşıma işlemlerini **her zaman** `noexcept` yapın. `std::vector` yeniden boyutlanırken elemanlarını yalnızca taşıma yapıcı `noexcept`'se *taşıır*; değilse güvenlik için *kopyalar*. Yani `noexcept` olmayan bir taşıma yapıcı, `vector` büyüdüğünde sessizce performans kaybettirir.

15.6 5. Taşıma Atama Operatörü (Move Assignment)

Taşıma atama, var olan bir nesneye bir rvalue atar (`e = makeTemp();`). Önce kendi eski kaynağını bırakır, sonra sağdaki geçicinin kaynağını çalar ve onu boşaltır. Kopya atamanın ucuz kardeşidir: bellek kopyalamak yerine sahiplik devreder.

Bilgi

Taşıma sonrası durum: `std::move` bir şeyi *taşımaz*; yalnızca nesneyi rvalue'ya çevirerek “kaynağını çalabilirsin” der. Gerçek taşımayı yukarıdaki fonksiyonlar yapar. Taşınan kaynak nesne yok *edilmez*; “geçerli ama belirsiz” durumda kalır. Onu tutarlı bir boş duruma getirmek (`data_ = nullptr`) şarttır ki yıkıcısı `delete[] nullptr` (güvenli) çağırılsın. Çıktıda `a`'nın kaynağı `d`'ye taşındıktan sonra `a.get()` “(bos)” döndürür.

Hangisi Ne Zaman?

Bir ifadenin kopya mı taşıma mı tetikleyeceğini sağ taraf belirler: sağ taraf bir **lvalue** (isimli, kalıcı nesne) ise *kopya*, bir **rvalue** (geçici, `std::move`'lu) ise *taşıma* çağrılır. `b = a` kopya, `b = std::move(a)` veya `b = makeTemp()` taşımadır. Hedefin yeni mi (yapıcı) yoksa var olan mı (atama) olduğu ise *yapıcı* ile *atama* operatörünü ayırır.

16. Üç, Beş ve Sıfır Kuralı

16.1 Üçün Kuralı (Rule of Three) — C++98

Eğer bir sınıfın şu üçünden *birini* elle yazmanız gerekiyorsa, büyük olasılıkla *üçünü de* yazmanız gerekir: **yıkıcı**, **kopya yapıcı**, **kopya atama operatörü**. Çünkü elle yönetilen bir kaynak (öbek belleği vb.) varsa, üçü de o kaynağı doğru ele almalıdır.

16.2 Beşin Kuralı (Rule of Five) — C++11

C++11 ile taşıma semantiği gelince, liste beşe çıktı. Elle kaynak yöneten bir sınıf genellikle şu **beşini** tanımlamalıdır:

#	Fonksiyon	İmza
1	Yıkıcı	~T()
2	Kopya yapıcı	T(const T&)
3	Kopya atama	T& operator=(const T&)
4	Taşıma yapıcı	T(T&&) noexcept
5	Taşıma atama	T& operator=(T&&) noexcept

Dikkat

Kritik kural: Bu beş fonksiyondan herhangi birini elle tanımlarsanız, derleyici diğer bazılarını otomatik üretmeyi *durdurabilir*. Örneğin bir yıkıcı yazarsanız, derleyici taşıma işlemlerini *üretmez* (ve kopya işlemleri “kullanımdan kaldırılmış” sayılır). Bu yüzden ya hepsini bilinçli tanımlayın, ya da hiçbirini (aşağıdaki Sıfır Kuralı).

16.3 Sıfırın Kuralı (Rule of Zero) — En İyi Yaklaşım

Modern C++’ın *tercih edilen* yaklaşımı, bu beş fonksiyonun **hiçbirini** yazmamaktır! Kaynakları doğrudan yönetmek yerine, zaten doğru yöneten türleri (`std::vector`, `std::string`, `std::unique_ptr`) üye olarak kullanın. O zaman derleyicinin ürettiği varsayılan özel üye fonksiyonlar *zaten doğru* çalışır.

```

1 #include <string>
2 #include <vector>
3 #include <memory>
4
5 // SIFIRIN KURALI: hiçbir özel üye fonksiyon yazmadık!
6 // Ciplak new/delete yok; kaynak yöneten uyeler kendileri halleder.
7 class User {
8     std::string name_;           // kendi kopya/tasımaInI yönetir
9     std::vector<int> scores_;    // kendi kopya/tasımaInI yönetir
10    std::unique_ptr<int> secret_; // tasınabilir, kopyalanamaz
11 public:
12    User(std::string name)
13        : name_(std::move(name)),
14          secret_(std::make_unique<int>(42)) {}
15
16    // Yıkıcı, kopya/tasıma... HİCBİRİNİ yazmadık.
17    // Derleyici hepsini DOĞRU şekilde üretir:
18    // - kopya: unique_ptr kopyalanamadığı için bu sInIf da kopyalanamaz
19    // - tasıma: uyelerin tasımaInI çağırır (otomatik, doğru, noexcept)
20 };

```

Bilgi

En iyi tavsiye: Sınıflarınızı Sıfır Kuralına göre tasarlayın. Ham kaynak yöneten (elle `new/delete` yapan) sınıflar yazmaktan kaçın; bunun yerine kaynağı bir `unique_ptr` veya konteyner içine sarın. Bu sayede Beşin Kuralı’nın tüm karmaşıklığı ve hata riski ortadan kalkar. Beşin Kuralı’nı yalnızca gerçekten düşük seviye bir kaynak sarmalayıcı (örneğin bir C API tanıtıcısı) yazarken uygulayın.

16.4 = default ve = delete ile Açık Kontrol

C++11, özel üye fonksiyonların üretimini açıkça kontrol etmenizi sağlar. `= default` derleyiciden varsayılanı üretmesini ister; `= delete` o fonksiyonu tamamen yasaklar.

```

1 class Example {
2 public:
3     Example() = default; // varsayilan yapiciIyiI uret
4
5     // Kopyalamayi acikca YASAKLA (ornegin tekil sahiplik icin)
6     Example(const Example&) = delete;
7     Example& operator=(const Example&) = delete;
8
9     // Tasimaya IZIN ver (varsayilanI uret)
10    Example(Example&&) noexcept = default;
11    Example& operator=(Example&&) noexcept = default;
12
13    virtual ~Example() = default; // varsayilan sanal yIkIcI
14 };

```

İpucu

= default, “derleyicinin ürettiği varsayılan davranış tam istediğim şey, ama niyetimi açıkça belirtiyorum” demektir — ayrıca bazı durumlarda (örneğin başka bir yapıcı tanımladığınızda kaybolan varsayılan yapıcıyı geri getirmek için) gereklidir. = delete ise kopyalamayı, istenmeyen dönüşümleri veya belirli aşırı yüklemeleri derleme zamanında yasaklamanın temiz yoludur.

17. Copy-and-Swap Deyimi

Copy-and-swap, kopya atama operatörünü hem *istisna güvenli* hem *kendine-atama güvenli* hem de kısa yazmanın zarif bir yoludur. Fikir: argümanı *değer olarak* al (böylece kopya otomatik oluşur), sonra mevcut nesneyle swap et. Bonus: tek bir atama operatörü hem kopya hem taşıma atamasını halleder.

```

1 #include <algorithm>
2 #include <cstring>
3 #include <utility>
4
5 class TextHolder {
6     char* data_ = nullptr;
7     std::size_t size_ = 0;
8 public:
9     TextHolder(const char* s = "")
10        : size_(std::strlen(s)), data_(new char[size_ + 1]) {
11        std::strcpy(data_, s);
12    }
13    ~TextHolder() { delete[] data_; }
14
15    TextHolder(const TextHolder& d)
16        : size_(d.size_), data_(new char[d.size_ + 1]) {
17        std::strcpy(data_, d.data_);
18    }
19    TextHolder(TextHolder&& d) noexcept { swap_with(d); } // bos'tan basla, takas et
20
21    // Uyeleri takas eden yardImcI (ADL icin ozgur swap kullanIr)
22    void swap_with(TextHolder& d) noexcept {
23        using std::swap;
24        swap(data_, d.data_);
25        swap(size_, d.size_);

```

```
26     }
27
28     // TEK atama operatoru: argumanI DEGER olarak alIr.
29     // - lvalue verilirse: kopya yapIcI cagrIlIr
30     // - rvalue verilirse: tasIma yapIcI cagrIlIr
31     // Sonra sadece takas ederiz. Kendine-atama ve istisna guvenligi BEDAVA.
32     TextHolder& operator=(TextHolder d) noexcept {
33         swap_with(d);
34         return *this;
35     } // 'd' (eski icerigimizi tasIyan) burada yok olur, eski kaynak temizlenir
36
37     const char* get() const { return data_; }
38 };
```

Bilgi

Copy-and-swap'ın zarafeti: argüman d *değer olarak* alındığı için, çağıran lvalue geçtiyse kopya yapıcı, rvalue geçtiyse taşıma yapıcı otomatik devreye girer. Böylece *tek* bir operator= hem kopya hem taşıma atamasını halleder. Kendine-atama kontrolüne gerek yoktur (kopya zaten oluşmuştur) ve güçlü istisna güvenliği sağlanır (takas noexcept'tir, kopya başarısız olursa nesne değişmeden kalır).

KISIM 6

Şablonlar ve Generic Programlama

18. Fonksiyon Şablonları

Şablonlar (templates), C++'ın *jenerik programlama* aracıdır: tek bir kod parçasını *farklı tipler* için yeniden yazmadan çalıştırmayı sağlar. Derleyici, kullanılan her tip için şablonu *somutlaştırır* (instantiate). Bu, `std::vector` veya `std::sort` gibi tüm STL'in temelidir.

```

1  #include <iostream>
2  #include <string>
3
4  // Fonksiyon sablonu: T herhangi bir tip olabilir
5  template <typename T>
6  T maximum(T a, T b) {
7      return (a > b) ? a : b;
8  }
9
10 // İki farklı tip parametresi
11 template <typename T, typename U>
12 auto add(T a, U b) -> decltype(a + b) { // donus tipini cikart
13     return a + b;
14 }
15
16 int main() {
17     // Derleyici T'yi argumanlardan CIKARIR (template argument deduction)
18     std::cout << maximum(3, 7) << "\n";           // T=int -> 7
19     std::cout << maximum(2.5, 1.5) << "\n";       // T=double -> 2.5
20     std::cout << maximum<std::string>("elma", "armut") << "\n"; // acik tip
21
22     // Karisik tipler: int + double -> double
23     std::cout << add(3, 4.5) << "\n";             // 7.5
24     return 0;
25 }

```

Bilgi

Derleyici `maximum(3, 7)` çağrısını görünce `T = int` için somut bir fonksiyon üretir; `maximum(2.5, 1.5)` için ayrı bir `double` sürümü üretir. Bu “sıfır maliyetli soyutlamadır”: elle yazılmış tipe özel koddan hiçbir çalışma zamanı farkı yoktur. Şablonlar tamamen derleme zamanında çözülür.

19. Sınıf Şablonları

Sınıflar da şablonlaştırılabilir. Aşağıda basit ama tam çalışan, jenerik bir yığın (stack) uygulaması var — `std::stack`'in nasıl çalıştığını gösterir.

```

1  #include <iostream>
2  #include <vector>
3  #include <stdexcept>
4
5  // Herhangi bir tip T tutabilen jenerik yigin
6  template <typename T>
7  class Stack {
8      std::vector<T> items_;
9  public:
10     void push(const T& value) { items_.push_back(value); }
11
12     void pop() {
13         if (empty()) throw std::out_of_range("Bos yigin");
14         items_.pop_back();
15     }
16
17     const T& top() const {
18         if (empty()) throw std::out_of_range("Bos yigin");
19         return items_.back();
20     }
21
22     bool empty() const { return items_.empty(); }
23     std::size_t size() const { return items_.size(); }
24 };
25
26 int main() {
27     Stack<int> numbers;
28     numbers.push(10); numbers.push(20); numbers.push(30);
29     std::cout << "Tepe: " << numbers.top() << ", boyut: " << numbers.size() << "\n";
30     numbers.pop();
31     std::cout << "Pop sonrasi tepe: " << numbers.top() << "\n";    // 20
32
33     // Ayni sablon, farkli tiple
34     Stack<std::string> words;
35     words.push("merhaba"); words.push("dunya");
36     std::cout << words.top() << "\n";    // dunya
37     return 0;
38 }

```

20. Değişken Sayıda Şablon (Variadic) ve Kat İfadeleri

Değişken sayıda şablon parametresi (variadic templates), keyfi sayıda argüman alan jenerik fonksiyonlar yazmanızı sağlar. C++17'nin *kat ifadeleri* (fold expressions) bunu son derece zarif kılar.

```

1  #include <iostream>
2
3  // C++17 kat ifadesi: tum argumanlari topla
4  template <typename... Args>
5  auto sum(Args... args) {
6      return (args + ...);    // kat ifadesi: ((a1 + a2) + a3) + ...
7  }

```

```
8
9 // Her argumani ekrana yazan variadic fonksiyon
10 template <typename... Args>
11 void print(const Args&... args) {
12     ((std::cout << args << " "), ...); // virgul kat ifadesi
13     std::cout << "\n";
14 }
15
16 int main() {
17     std::cout << sum(1, 2, 3, 4, 5) << "\n"; // 15
18     std::cout << sum(1.5, 2.5) << "\n"; // 4.0
19     print("Sayilar:", 1, 2.5, 'X', "bitti"); // karisik tipler
20     return 0;
21 }
```

İpucu

Kat ifadeleri ((args + ...)) C++17 ile geldi ve variadic şablonları okunması zor özyinelemeli yardımcılardan kurtardı. (args + ...) sağ kat, (... + args) sol kattır. Virgül operatörüyle ((ifade, ...)) her argüman için bir yan etki (örneğin yazdırma) çalıştırabilirsiniz. Bu, modern jenerik kodun temel araçlarından biridir.

KISIM 7

STL: Konteynerler, İteratörler ve Algoritmalar

21. Standart Konteynerler

STL (Standard Template Library), C++'ın hazır veri yapıları ve algoritmalarından oluşan çekirdeğidir. Konteynerler, verileri farklı şekillerde saklar; her birinin kendine özgü performans karakteristiği vardır.

Konteyner	Yapı	Tipik Erişim
<code>vector</code>	Dinamik dizi (bitişik bellek)	Sonda $\mathcal{O}(1)$, indeks $\mathcal{O}(1)$
<code>deque</code>	Çift uçlu kuyruk	İki uçta $\mathcal{O}(1)$
<code>list</code>	Çift bağlı liste	Ekleme/silme $\mathcal{O}(1)$
<code>map</code>	Sıralı (kırmızı-siyah ağaç)	Arama $\mathcal{O}(\log n)$
<code>unordered_map</code>	Hash tablosu	Arama ortalama $\mathcal{O}(1)$
<code>set</code>	Sıralı benzersiz küme	$\mathcal{O}(\log n)$

```

1 #include <iostream>
2 #include <vector>
3 #include <map>
4 #include <unordered_map>
5 #include <set>
6 #include <string>
7
8 int main() {
9     // vector: en çok kullanılan konteyner (dinamik dizi)
10    std::vector<int> numbers{5, 3, 8, 1};
11    numbers.push_back(9);
12    std::cout << "vector[2] = " << numbers[2] << ", boyut = " << numbers.size() << "\n";
13
14    // map: sıralı anahtar-değer (otomatik sıralı gezilir)
15    std::map<std::string, int> ages;
16    ages["Ada"] = 36;
17    ages["Alan"] = 41;

```

```

18  ages["Grace"] = 45;
19  for (const auto& [name, age] : ages)           // yapisal baglama (C++17)
20      std::cout << name << ": " << age << "\n"; // alfabetik sirali

21
22  // unordered_map: hash tabanlı (siralamaz ama ortalama O(1))
23  std::unordered_map<std::string, double> prices;
24  prices["kalem"] = 5.0;
25  prices["defter"] = 12.5;
26  if (auto it = prices.find("kalem"); it != prices.end()) // if-init (C++17)
27      std::cout << "kalem fiyatı: " << it->second << "\n";
28
29  // set: benzersiz, sirali
30  std::set<int> unique{3, 1, 4, 1, 5, 9, 2, 6, 5}; // tekrarlar elenir
31  std::cout << "Benzersiz eleman sayısı: " << unique.size() << "\n";
32  return 0;
33 }

```

22. İteratörler

İteratör (iterator), bir konteynerin elemanları üzerinde gezinmek için kullanılan, işaretçiye genelleştiren bir soyutlamadır. `begin()` ilk elemanı, `end()` son elemanın *bir sonrasını* gösterir. Tüm STL algoritmaları iteratörler üzerinden çalışır — bu sayede algoritmalar konteyner tipinden bağımsızdır.

```

1  #include <iostream>
2  #include <vector>
3  #include <list>
4
5  int main() {
6      std::vector<int> v{10, 20, 30, 40};
7
8      // Klasik iterator dongusu
9      for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)
10         std::cout << *it << " ";           // *it: iteratorun gosterdigi eleman
11     std::cout << "\n";
12
13     // auto ile cok daha kisa
14     for (auto it = v.begin(); it != v.end(); ++it)
15         std::cout << *it << " ";
16     std::cout << "\n";
17
18     // Ters iterator (sondan basa)
19     for (auto it = v.rbegin(); it != v.rend(); ++it)
20         std::cout << *it << " ";           // 40 30 20 10
21     std::cout << "\n";
22
23     // Iteratorler farkli konteynerlerde ayni sekilde calisir
24     std::list<char> chars{'a', 'b', 'c'};
25     for (auto it = chars.begin(); it != chars.end(); ++it)
26         std::cout << *it << " ";
27     std::cout << "\n";
28     return 0;
29 }

```

Bilgi

`end()` geçerli bir elemanı göstermez; son elemanın bir sonrasını (“geçmiş-son”, past-the-end) gösteren bir nöbetçidir. Döngü koşulu bu yüzden `it != end()` biçimindedir. Bir iteratörü `end()`’e ulaştıktan sonra dereference etmek (`*it`) tanımsız davranıştır. Ayrıca konteyneri değiştirmek (örneğin `vector`’e ekleme yapmak) mevcut iteratörleri geçersiz kılabilir — buna dikkat edin.

23. Algoritmalar

`<algorithm>` başlığı, iteratör aralıkları üzerinde çalışan onlarca hazır algoritma sunar: sıralama, arama, dönüştürme, sayma, biriktirme. Kendi döngülerinizi yazmak yerine bunları kullanmak hem daha kısa hem daha az hatalıdır.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <numeric>
5
6  int main() {
7      std::vector<int> v{5, 2, 8, 1, 9, 3};
8
9      // sort: sirala (varsayılan artan)
10     std::sort(v.begin(), v.end());
11     for (int x : v) std::cout << x << " ";      // 1 2 3 5 8 9
12     std::cout << "\n";
13
14     // Lambda ile özel ölçüt: azalan sirala
15     std::sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
16     for (int x : v) std::cout << x << " ";      // 9 8 5 3 2 1
17     std::cout << "\n";
18
19     // find: bir eleman ara
20     if (auto it = std::find(v.begin(), v.end(), 5); it != v.end())
21         std::cout << "5 bulundu, konum: " << (it - v.begin()) << "\n";
22
23     // count_if: koşulu sağlayanları say
24     int evenCount = std::count_if(v.begin(), v.end(),
25                                   [](int x){ return x % 2 == 0; });
26     std::cout << "Çift sayı adedi: " << evenCount << "\n";    // 2
27
28     // transform: her elemanı donustur (karesini al)
29     std::vector<int> squares(v.size());
30     std::transform(v.begin(), v.end(), squares.begin(),
31                   [](int x){ return x * x; });
32
33     // accumulate: hepsini birlestir (topla)
34     int total = std::accumulate(v.begin(), v.end(), 0);
35     std::cout << "Toplam: " << total << "\n";    // 28
36
37     // max_element / min_element
38     auto maxIt = std::max_element(v.begin(), v.end());
39     std::cout << "En büyük: " << *maxIt << "\n";    // 9
40     return 0;
41 }
```

İpucu

“Ham döngü yazmadan önce bir STL algoritması var mı?” diye sorun. `std::sort`, `std::find`, `std::accumulate`, `std::transform`, `std::count_if`, `std::any_of/all_of` gibi algoritmalar niyetinizi net ifade eder ve hata olasılığını azaltır. C++20 ile gelen *ranges* (`std::ranges::sort(v)`) bunları daha da kısaltır — artık `begin()/end()` çifti yazmanıza bile gerek kalmaz.

KISIM 8

Lambda ve Fonksiyonel Programlama

24. Lambda İfadeleri

Lambda ifadeleri, kod içinde *yerinde* isimsiz fonksiyonlar tanımlamanızı sağlar. STL algoritmalarına ölçüt geçmek, geri çağırımlar ve kısa yardımcı işlemler için idealdir. Genel biçim: [yakalama] (parametreler) { gövde }.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  int main() {
6      // En basit lambda
7      auto greet = []{ std::cout << "Merhaba lambda!\n"; };
8      greet();
9
10     // Parametrelili ve donusluu
11     auto add = [](int a, int b) { return a + b; };
12     std::cout << add(3, 4) << "\n";    // 7
13
14     // YAKALAMA (capture): disaridaki degiskenleri kullan
15     int factor = 10;
16     auto multiply = [factor](int x) { return x * factor; }; // deger ile yakala
17     std::cout << multiply(5) << "\n";    // 50
18
19     // Referansla yakalama: disaridaki degiskeni DEGISTIR
20     int counter = 0;
21     auto increment = [&counter]{ ++counter; };
22     increment(); increment(); increment();
23     std::cout << "Sayac: " << counter << "\n";    // 3
24
25     // [=] tumunu degerle, [&] tumunu referansla yakalar
26     int a = 1, b = 2;
27     auto sumAll = [=]{ return a + b; };    // a,b kopyalanir
28     std::cout << sumAll() << "\n";    // 3
29
30     // mutable: deger-yakalanan kopyayi lambda icinde degistirmeye izin ver
31     auto gen = [n = 0]() mutable { return ++n; };    // baslatmali yakalama (C++14)
32     std::cout << gen() << gen() << gen() << "\n";    // 123
33
34     // Jenerik lambda (C++14): auto parametre
35     auto genericMax = [](auto x, auto y){ return x > y ? x : y; };

```

```

36     std::cout << genericMax(3, 7) << " " << genericMax(2.5, 1.5) << "\n";
37     return 0;
38 }

```

Dikkat

Referansla yakalamada ([&]) ömür yönetimine dikkat edin: lambda, yakaladığı referanstan *daha uzun* yaşarsa (örneğin bir fonksiyondan döndürülüp sonra çağrılırsa), sarkan referans oluşur ve tanımsız davranışa yol açar. Bir lambda'yı içinde tanımlandığı kapsamdan dışarı taşıyacaksanız, referans yerine *değerle* ([=] veya açık kopya) yakalayın.

25. std::function ve Fonksiyon Nesneleri

std::function, herhangi bir çağrılabilir nesneyi (lambda, serbest fonksiyon, functor, üye fonksiyon) tek tip bir sarmalayıcıda tutan genel amaçlı bir tiptir. Çağrılabilir nesneleri veri gibi saklamak, geçirmek ve kaydetmek (örneğin geri çağırma tabloları) için kullanılır.

```

1  #include <iostream>
2  #include <functional>
3  #include <vector>
4  #include <map>
5
6  int freeFunction(int x) { return x + 1; }
7
8  int main() {
9      // std::function farklı çağrılabilirleri AYNI tipte tutar
10     std::function<int(int)> f;
11
12     f = freeFunction;                // serbest fonksiyon
13     std::cout << f(10) << "\n";      // 11
14
15     f = [](int x){ return x * x; };  // lambda
16     std::cout << f(5) << "\n";       // 25
17
18     // Çağrılabilirleri bir kaptaki sakla
19     std::vector<std::function<int(int)>> operations = {
20         [](int x){ return x + 1; },
21         [](int x){ return x * 2; },
22         [](int x){ return x * x; }
23     };
24     for (auto& op : operations)
25         std::cout << op(5) << " ";   // 6 10 25
26     std::cout << "\n";
27
28     // Komut tablosu: string -> islev (dispatch table deseni)
29     std::map<std::string, std::function<double(double,double)>> calc = {
30         {"topla",    [](double a, double b){ return a + b; }},
31         {"cikar",    [](double a, double b){ return a - b; }},
32         {"carp",     [](double a, double b){ return a * b; }}
33     };
34     std::cout << "3 carp 4 = " << calc["carp"](3, 4) << "\n"; // 12
35     return 0;
36 }

```


İpucu

`std::function` esnektir ama küçük bir çalışma zamanı maliyeti (tip silme, olası yığın tahsisi, dolaylı çağrı) taşır. Performans kritik sıcak döngülerde, tipi bilinen bir lambda'yı doğrudan (veya bir şablon parametresi olarak) kullanmak daha hızlıdır. `std::function`'ı asıl gücü olan *heterojen çağrılabilirleri saklama* senaryolarında (geri çağırma listeleri, komut tabloları) kullanın.

KISIM 9

İstisna Yönetimi (Exceptions)

26. try, catch ve throw

İstisnalar (exceptions), çalışma zamanı hatalarını — fonksiyonun normal dönüş değerini kirlletmeden — bildirmenin ve ele almanın yapılandırılmış bir yoludur. Bir hata oluştuğunda `throw` ile istisna *fırlatılır*; çağrı yığını, onu yakalayacak bir `catch` bulunana kadar *çözülür* (stack unwinding) — bu sırada tüm yerel nesnelerin yıkıcıları çalışır.

```
1  #include <iostream>
2  #include <stdexcept>
3  #include <string>
4
5  double divide(double a, double b) {
6      if (b == 0)
7          throw std::invalid_argument("Sifira bolme!"); // istisna firlat
8      return a / b;
9  }
10
11 int main() {
12     try {
13         std::cout << divide(10, 2) << "\n"; // 5
14         std::cout << divide(10, 0) << "\n"; // istisna firlatir
15         std::cout << "Bu satir CALISMAZ\n";
16     }
17     catch (const std::invalid_argument& e) { // ozel istisna tipini yakala
18         std::cout << "Gecersiz arguman: " << e.what() << "\n";
19     }
20     catch (const std::exception& e) { // genel taban: her std istisnasi
21         std::cout << "Genel hata: " << e.what() << "\n";
22     }
23     catch (...) { // her seyi yakalayan son savunma
24         std::cout << "Bilinmeyen hata\n";
25     }
26
27     std::cout << "Program devam ediyor\n"; // catch sonrasi normal akis
28     return 0;
29 }
```

Bilgi

catch blokları *en özelden en genele* sıralanmalıdır: önce `std::invalid_argument`, sonra taban sınıfı `std::exception`, en sonda `catch(...)`. İstisnaları **her zaman referansla** (`const std::exception&`) yakalayın; değerle yakalamak nesne kırılmasına (slicing) yol açar ve polimorfik `what()` mesajını kaybettirir. `<stdexcept>` başlığı hazır istisna tipleri sunar: `logic_error`, `runtime_error`, `out_of_range`, `invalid_argument` vb.

27. Özel İstisnalar ve İstisna Güvenliği

Kendi istisna tiplerinizi `std::exception`'dan türeterek tanımlayın. Ayrıca, istisnalar RAII ile birlikte kullanıldığında (Kısım 4) kod *istisna güvenli* olur: bir istisna atıldığında kaynaklar otomatik temizlenir.

```

1  #include <iostream>
2  #include <stdexcept>
3  #include <string>
4  #include <memory>
5
6  // Özel istisna tipi: std::runtime_error'dan turet
7  class NetworkError : public std::runtime_error {
8      int errorCode_;
9  public:
10     NetworkError(const std::string& message, int code)
11         : std::runtime_error(message), errorCode_(code) {}
12     int code() const { return errorCode_; }
13 };
14
15 void connect(bool fail) {
16     // RAII kaynagi: istisna atilrsa BILE otomatik temizlenir
17     auto resource = std::make_unique<int>(42);
18
19     if (fail)
20         throw NetworkError("Baglanti reddedildi", 503);
21
22     std::cout << "Baglanti basarili, kaynak = " << *resource << "\n";
23 } // 'resource' kapsam disinda otomatik silinir (istisna olsa da olmasa da)
24
25 int main() {
26     try {
27         connect(false); // basarili
28         connect(true);  // istisna firlatir
29     }
30     catch (const NetworkError& e) {
31         std::cout << "Ag hatasi [" << e.code() << "]: " << e.what() << "\n";
32     }
33     return 0;
34 }

```

Dikkat

Yıkıcılardan istisna atmayın. C++11'den beri yıkıcılar örtük olarak `noexcept`'tir; bir yıkıcı istisna atarsa (özellikle yığın çözülürken başka bir istisna zaten aktifse) program `std::terminate` ile derhal sonlanır. Ayrıca istisnaları normal akış kontrolü için kullanmayın; onlar *istisnai* (beklenmedik hata) durumlar içindir. Sık gerçekleşen “hata” durumları için `std::optional` veya C++23 `std::expected` daha uygundur.

KISIM 10

Önemli Anahtar Kelimeler

28. explicit: İstenmeyen Dönüşümleri Engelleme

Tek argümanlı bir yapıcı, C++'ta *örtük (implicit) dönüşüm* olarak kullanılabilir — bu genellikle istenmeyen ve şaşırtıcı sonuçlar doğurur. `explicit` anahtar kelimesi bu örtük dönüşümü yasaklar.

```

1 #include <iostream>
2
3 class Meter {
4     double value_;
5 public:
6     // explicit OLMASAYDI: 'Metre m = 5.0;' ve hatta fonksiyona '5.0'
7     // gecmek sessizce Metre'ye donusurdu -- tehlikeli.
8     explicit Meter(double d) : value_(d) {}
9     double value() const { return value_; }
10 };
11
12 void print(Meter m) { std::cout << m.value() << " metre\n"; }
13
14 int main() {
15     Meter a(5.0);           // OK: dogrudan baslatma
16     // Meter b = 5.0;       // HATA (explicit): ortuk donusum yasak
17     Meter c = Meter(5.0);   // OK: acik donusum
18     // print(10.0);         // HATA: 10.0 sessizce Metre'ye donmez
19     print(Meter(10.0));     // OK: acik
20     return 0;
21 }
```

İpucu

Kural: Tek argümanla çağrılabilen yapıcıları — özellikle bir birim/tip sarmalayıcısıysa — neredeyse her zaman `explicit` yapın. Örtük dönüşümü yalnızca gerçekten istediğinizde (örneğin `std::string`'in `const char*`'tan dönüşümü) açık bırakın. C++'ta “varsayılan olarak `explicit`” iyi bir alışkanlıktır.

29. const ve constexpr

`const`, “bu değer değiştirilemez” der (çalışma zamanı sabiti). `constexpr`, “bu değer/fonksiyon derleme zamanında hesaplanabilir” der. `constexpr` çok daha güçlüdür ve performans için değerlidir.

```

1  #include <iostream>
2  #include <array>
3
4  // constexpr fonksiyon: derleme zamanında hesaplanabilir
5  constexpr int factorial(int n) {
6      return (n <= 1) ? 1 : n * factorial(n - 1);
7  }
8
9  class Circle {
10     double radius_;
11 public:
12     constexpr Circle(double r) : radius_(r) {}
13
14     // const üye fonksiyon: nesneyi DEĞİSTİRMEZ (this üzerinden yazamaz)
15     double area() const { return 3.14159 * radius_ * radius_; }
16 };
17
18 int main() {
19     const int x = 10;           // çalışma zamanı sabiti
20     // x = 20;                 // HATA: const
21
22     constexpr int f5 = factorial(5); // DERLEME zamanında hesaplanır: 120
23     std::array<int, factorial(4)> arr; // constexpr, sablon argümanı olabilir (24)
24     std::cout << "5! = " << f5 << ", dizi boyutu = " << arr.size() << "\n";
25
26     const Circle d(2.0);
27     std::cout << "Alan = " << d.area() << "\n"; // const nesnede const üye çağrılır
28     return 0;
29 }

```

Bilgi

const üye fonksiyon (`alan() const`), o fonksiyonun nesnenin durumunu değiştirmeyeceğine söz verir; bu sayede `const` nesneler üzerinde çağrılabilir. Nesnenin durumunu değiştirmeyen *her* üye fonksiyonu `const` yapmak (“const doğruluğu”, `const-correctness`) iyi bir tasarım disiplini. C++20’de `constexpr` (mutlaka derleme zamanı) ve `constexpr` de eklendi.

30. noexcept: İstisna Atmama Garantisi

`noexcept`, bir fonksiyonun istisna *atmayacağını* bildirir. Bu hem bir optimizasyon ipucudur hem de — daha önce gördüğümüz gibi — taşıma semantiği için kritik bir sözleşmedir. `noexcept` bir fonksiyon yine de istisna atarsa, program derhal `std::terminate` ile sonlanır.

```

1  #include <iostream>
2  #include <vector>
3
4  int safe_add(int a, int b) noexcept { // asla istisna atmaz
5      return a + b;
6  }
7
8  // Tasıma işlemleri noexcept OLMALI (vector optimizasyonu için)
9  struct Movable {
10     int* data;
11     Movable(Movable&& d) noexcept : data(d.data) { d.data = nullptr; }

```

```

12     Movable& operator=(Movable&&) noexcept = default;
13     ~Movable() { delete data; }
14     Movable() : data(new int(0)) {}
15 };
16
17 int main() {
18     // noexcept operatoru: bir ifadenin noexcept olup olmadigInI sorgular
19     std::cout << std::boolalpha
20         << noexcept(safe_add(1, 2)) << "\n";    // true
21
22     // vector, noexcept tasIma yapIcIsI olan tipleri buyurken TASIR (kopyalamaz)
23     std::vector<Movable> v;
24     v.reserve(2);
25     v.emplace_back();
26     v.emplace_back();    // yeniden tahsiste noexcept tasIma kullanIlIr
27     return 0;
28 }

```

Dikkat

Bir fonksiyonu noexcept işaretleyip sonra istisna atarsanız, catch ile yakalayamazsınız — program `std::terminate` çağırarak anında ölür. Bu yüzden noexcept'i yalnızca gerçekten istisna atmayacağından *emin* olduğunuz fonksiyonlarda kullanın. Yıkıcılar C++11'den beri örtük olarak noexcept'tir; yıkıcıdan istisna atmak neredeyse her zaman bir tasarım hatasıdır.

31. static, mutable ve friend

31.1 static Üyeler

```

1  #include <iostream>
2
3  class Counter {
4      static int total_;    // TUM nesneler icin ORTAK tek bir degisken
5      int id_;
6  public:
7      Counter() : id_(++total_) {}
8      static int count() { return total_; }    // static uye fonksiyon (this yok)
9      int id() const { return id_; }
10 };
11 int Counter::total_ = 0;    // static uye sInIf dISInda TANIMLANMALI
12
13 int main() {
14     Counter a, b, c;
15     std::cout << "Toplam nesne: " << Counter::count() << "\n";    // 3
16     std::cout << "c'nin id'si: " << c.id() << "\n";    // 3
17     return 0;
18 }

```

31.2 mutable Üyeler

mutable, bir üyenin const bir nesnede bile değiştirilebilmesini sağlar. Genellikle önbellekleme, kilit veya sayaç gibi “mantıksal durumu” değiştirmeyen yardımcı üyeler için kullanılır.

```

1  #include <iostream>

```

```

2
3 class Calculator {
4     mutable int callCount_ = 0;    // const uyede bile degisebilir
5     int value_;
6 public:
7     Calculator(int d) : value_(d) {}
8
9     // const fonksiyon ama mutable uyeyi degistirebilir
10    int square() const {
11        ++callCount_;               // mutable sayesinde OK
12        return value_ * value_;
13    }
14    int callCount() const { return callCount_; }
15 };
16
17 int main() {
18     const Calculator h(5);          // const NESNE
19     std::cout << h.square() << " " << h.square() << "\n";
20     std::cout << "Cagri: " << h.callCount() << "\n";    // 2
21     return 0;
22 }

```

31.3 friend Fonksiyonlar

friend, bir fonksiyona veya sınıfa, başka bir sınıfın private üyelerine erişim ayrıcalığı verir. En yaygın kullanımı, birazdan göreceğimiz akış operatörünü (operator«) aşırı yüklemektir.

```

1 #include <iostream>
2
3 class Point {
4     int x_, y_;    // private
5 public:
6     Point(int x, int y) : x_(x), y_(y) {}
7     // friend: bu serbest fonksiyon private üyelere erisebilir
8     friend std::ostream& operator<<(std::ostream& os, const Point& p);
9 };
10
11 std::ostream& operator<<(std::ostream& os, const Point& p) {
12     return os << "(" << p.x_ << ", " << p.y_ << ")";    // private erisim
13 }
14
15 int main() {
16     Point p(3, 4);
17     std::cout << p << "\n";    // (3, 4)
18     return 0;
19 }

```


KISIM 11

Operatör Aşırı Yükleme

32. Operatör Aşırı Yüklemenin Temelleri

C++, kendi tiplerinizin +, ==, [], « gibi operatörlerle nasıl davranacağını tanımlamanıza izin verir. Bu, kendi tiplerinizi yerleşik tipler kadar doğal kullanmanızı sağlar. Ancak *dikkatli* kullanılmalıdır: operatörler sezgisel anlamlarını korumalıdır (+ toplama gibi davranmalı, sürpriz yapmamalı).

Bir matematiksel vektör (2B) sınıfı üzerinden operatörlerin çoğunu görelim:

```

1  #include <iostream>
2
3  class Vec2 {
4      double x_, y_;
5  public:
6      Vec2(double x = 0, double y = 0) : x_(x), y_(y) {}
7      double x() const { return x_; }
8      double y() const { return y_; }
9
10     // --- Bileşik atama (uye fonksiyon olarak yazmak GELENEKTİR) ---
11     Vec2& operator+=(const Vec2& r) { x_ += r.x_; y_ += r.y_; return *this; }
12     Vec2& operator-=(const Vec2& r) { x_ -= r.x_; y_ -= r.y_; return *this; }
13     Vec2& operator*=(double s)      { x_ *= s;    y_ *= s;    return *this; }
14
15     // --- Birli (unary) operatorler ---
16     Vec2 operator-() const { return Vec2(-x_, -y_); } // negatif
17
18     // --- Karşılasmı (uye) ---
19     bool operator==(const Vec2& r) const { return x_ == r.x_ && y_ == r.y_; }
20     bool operator!=(const Vec2& r) const { return !(*this == r); }
21
22     friend std::ostream& operator<<(std::ostream&, const Vec2&);
23 };
24
25 // --- İkili aritmetik: SERBEST fonksiyon olarak yazmak GELENEKTİR ---
26 // (Simetri için: 'a + b' ve donusumler her iki operand için de çalışır)
27 Vec2 operator+(Vec2 l, const Vec2& r) { return l += r; } // l KOPYA -> += -> don
28 Vec2 operator-(Vec2 l, const Vec2& r) { return l -= r; }
29 Vec2 operator*(Vec2 v, double s)      { return v *= s; }
30 Vec2 operator*(double s, Vec2 v)      { return v *= s; } // s * v de çalışır
31
32 std::ostream& operator<<(std::ostream& os, const Vec2& v) {
33     return os << "(" << v.x_ << ", " << v.y_ << ")";
34 }

```

```

35
36 int main() {
37     Vec2 a(1, 2), b(3, 4);
38     std::cout << "a + b   = " << (a + b) << "\n";      // (4, 6)
39     std::cout << "b - a   = " << (b - a) << "\n";      // (2, 2)
40     std::cout << "a * 2   = " << (a * 2) << "\n";      // (2, 4)
41     std::cout << "3 * a   = " << (3 * a) << "\n";      // (3, 6)
42     std::cout << "-a      = " << (-a) << "\n";          // (-1, -2)
43     std::cout << "a == b? " << std::boolalpha << (a == b) << "\n"; // false
44
45     a += b;                                           // bileşik atama
46     std::cout << "a += b  -> " << a << "\n";          // (4, 6)
47     return 0;
48 }

```

Bilgi

Üye mi, serbest fonksiyon mu? Genel kurallar: bileşik atama ($+=$, $-=$) ve nesnenin durumunu değiştiren operatörler **üye** olmalıdır. Simetrik ikili operatörler ($+$, $-$, $==$) **serbest fonksiyon** olmalıdır — böylece sol operand da örtük dönüşüme uğrayabilir. Kalıplaşmış deyim: `operator+`'yı, üye `operator+=` cinsinden yazın (yukarıdaki gibi). Bu, kod tekrarını önler ve tutarlılığı garanti eder.

33. Karşılaştırma ve Uzay Gemisi Operatörü (C++20)

C++20 öncesinde, bir tipi tam sıralanabilir yapmak için altı operatörü ($==$, $!=$, $<$, $<=$, $>$, $>=$) elle yazmanız gerekiyordu. C++20'nin **üç yönlü karşılaştırma operatörü** ($<=>$, “spaceship”) bunların hepsini tek satırda üretir.

```

1  #include <compare>
2  #include <iostream>
3
4  struct Version {
5      int major, minor, patch;
6
7      // Tek satır: derleyici ==, !=, <, <=, >, >= HEPSİNİ üretir
8      // Üyeler sırayla (once ana, sonra ara, sonra yama) karşılaştırılır
9      auto operator<=>(const Version&) const = default;
10 };
11
12 int main() {
13     Version a{1, 2, 0}, b{1, 3, 0};
14
15     std::cout << std::boolalpha;
16     std::cout << "a < b : " << (a < b) << "\n";      // true (ara: 2 < 3)
17     std::cout << "a == b: " << (a == b) << "\n";      // false
18     std::cout << "a >= b: " << (a >= b) << "\n";      // false
19
20     // <=> doğrudan kullanımı: sıralama sonucu döner
21     auto result = a <=> b;
22     if (result < 0)     std::cout << "a küçük\n";
23     else if (result > 0) std::cout << "a büyük\n";
24     else                std::cout << "esit\n";
25     return 0;
26 }

```

```
1 g++ -std=c++20 spaceship.cpp -o spaceship
```

İpucu

auto operator<=>(const T&) const = default; tek satırı, tipi tam sıralanabilir yapar ve std::sort, std::set, std::map ile doğrudan kullanılabilir hale getirir. Dönüş tipi karşılaştırma kategorisini belirtir: std::strong_ordering (tam sıra), std::weak_ordering (eşdeğerlik), std::partial_ordering (örn. NaN içeren float). = default ile derleyici en uygununu seçer.

34. Özel Operatörler: [], (), ve Dönüşüm

34.1 Alt Simge Operatörü []

```
1 #include <iostream>
2 #include <stdexcept>
3
4 class Array {
5     int* data_;
6     std::size_t size_;
7 public:
8     Array(std::size_t n) : data_(new int[n]{}), size_(n) {}
9     ~Array() { delete[] data_; }
10    Array(const Array&) = delete; // basitlik icin kopyayı kapat
11    Array& operator=(const Array&) = delete;
12
13    // İki asIrI yuikleme: degistirilebilir ve salt-okunur (const) erisim
14    int& operator[](std::size_t i) {
15        if (i >= size_) throw std::out_of_range("indeks");
16        return data_[i]; // referans dondurur -> a[i] = 5 calIsIr
17    }
18    const int& operator[](std::size_t i) const {
19        if (i >= size_) throw std::out_of_range("indeks");
20        return data_[i];
21    }
22    std::size_t size() const { return size_; }
23 };
24
25 int main() {
26     Array a(5);
27     for (std::size_t i = 0; i < a.size(); ++i)
28         a[i] = static_cast<int>(i * i); // operator[] (non-const) -> atama
29     for (std::size_t i = 0; i < a.size(); ++i)
30         std::cout << a[i] << " "; // 0 1 4 9 16
31     std::cout << "\n";
32     return 0;
33 }
```

34.2 Fonksiyon Çağrı Operatörü () — Fonksiyon Nesneleri

operator() aşırı yüklendiğinde, nesne bir fonksiyon gibi çağrılabilir (functor / function object) hale gelir. STL algoritmaları ve lambdaların temelinde bu yatar.

```
1 #include <iostream>
```

```

2 #include <vector>
3 #include <algorithm>
4
5 // Durum tutabilen bir fonksiyon nesnesi (functor)
6 class Multiplier {
7     int coefficient_;
8 public:
9     explicit Multiplier(int k) : coefficient_(k) {}
10    int operator()(int x) const { return x * coefficient_; } // çağrılabilir
11 };
12
13 int main() {
14     Multiplier triple(3);
15     std::cout << triple(10) << "\n"; // 30 -- nesneyi fonksiyon gibi çağır
16
17     std::vector<int> v = {1, 2, 3, 4};
18     // Functor'u STL algoritmasına geç
19     std::transform(v.begin(), v.end(), v.begin(), Multiplier(10));
20     for (int x : v) std::cout << x << " "; // 10 20 30 40
21     std::cout << "\n";
22
23     // Lambda: aslında derleyicinin ürettiği isimsiz bir functor'dur
24     auto add = [addend = 100](int x) { return x + addend; };
25     std::cout << add(5) << "\n"; // 105
26     return 0;
27 }

```

Bilgi

Bir lambda ifadesi, aslında derleyicinin arka planda ürettiği, operator() aşırı yüklenmiş isimsiz bir sınıftır. Yakalama listesi ([topla = 100]) o sınıfın üye değişkenlerine dönüşür. Bu yüzden functor'ları anlamak, lambdaların nasıl çalıştığını anlamaktır.

34.3 Dönüşüm Operatörü

Bir sınıfı başka bir tipe *örtük veya açık* dönüştürmek için dönüşüm operatörü tanımlanır. `explicit` ile açık dönüşüm zorunlu kılınabilir.

```

1 #include <iostream>
2
3 class Fraction {
4     int numerator_, denominator_;
5 public:
6     Fraction(int p, int q) : numerator_(p), denominator_(q) {}
7
8     // double'a ÖRTÜK donusum operatoru
9     operator double() const {
10         return static_cast<double>(numerator_) / denominator_;
11     }
12     // Bir sInIfIn 'bool baglamI'nda kullanIlmasI icin explicit tercih edilir
13     explicit operator bool() const { return denominator_ != 0; }
14 };
15
16 int main() {
17     Fraction half(1, 2);
18     double d = half; // örtük donusum -> 0.5
19     std::cout << "double: " << d << "\n";
20     std::cout << "1.5 + yarım = " << (1.5 + half) << "\n"; // 2.0

```

```
21  
22     if (half) std::cout << "gecerli kesir\n";    // explicit operator bool  
23     return 0;  
24 }
```

Dikkat

Örtük dönüşüm operatörleri (operator double() gibi) güçlüdür ama tehlikelidir: beklenmedik yerlerde sessizce devreye girip belirsizliğe ve hatalara yol açabilirler. Özellikle operator bool()'u neredeyse her zaman explicit yapın; aksi halde sınıfınız kazara tam sayı aritmetiğine veya karşılaştırmalara karışabilir. Genel tavsiye: dönüşüm operatörlerini idareli ve mümkünse explicit kullanın.

KISIM 12

Bellek Optimizasyonu: Yanlışlar ve Doğruları

Bellek optimizasyonu, C++'ın en çok “sezgiyle” yapılp en çok yanlış yapılan alanıdır. Çoğu program yavaşlar; çünkü hesaplama pahalı değil, *bellek erişimi* pahalıdır: gereksiz tahsisler (allocation), gizli kopyalar, önbellek (cache) ıskalamaları ve parçalanmış (fragmented) bellek düzeni. Bu kısım, en yaygın hataları “**Yanlış**” / “**Doğru**” çiftleri hâlinde gösterir. Her örnekte önce naif (ve genelde çalışan ama yavaş) sürümü, ardından ölçülebilir biçimde daha hızlı doğru sürümü göreceksiniz.

Bilgi

Optimizasyonun altın kuralı: önce ölç. Sezgiye güvenmeyin; darboğazı bir profilleyici (profiler) ile bulun. Bellek için: valgrind --tool=massif (öbek kullanımı), heaptrack (tahsis sayısı ve kaynağı), perf stat -e cache-misses (önbellek ıskalaması). Optimize ettiğiniz her şeyi *öncesi/sonrası* ölçün. “Erken optimizasyon” kadar zararlı olan tek şey, *ölçmeden* optimizasyondur.

35. Gereksiz Kopyalar ve Yeniden Tahsisler

En sık ve en pahalı hata, `std::vector`'ün kaç eleman alacağını bilerek ama yine de onu boş başlatıp tek tek büyütme. `vector` dolunca kapasitesini genellikle iki katına çıkarır: *yeni* bir bellek bloğu ayırır, tüm mevcut elemanları oraya taşır/kopyalar ve eskisini serbest bırakır. N eleman eklerken bu, $\log_2 N$ kez yeniden tahsis ve toplam $\mathcal{O}(N)$ ekstra kopya demektir.

35.1 Yanlış: reserve'süz büyüyen vektör

```

1 #include <vector>
2 #include <string>
3
4 std::vector<std::string> build_wrong(int n) {
5     std::vector<std::string> result;           // kapasite = 0
6     for (int i = 0; i < n; ++i)
7         result.push_back("oge-" + std::to_string(i));
8     // n=1'000'000 için: vektör ~20 kez yeniden tahsis edilir ve
9     // her seferinde TUM mevcut string'ler yeni bloğa TASINIR.
10    return result;
11 }
```

35.2 Doğru: kapasiteyi önceden ayır (reserve) + emplace

```

1 #include <vector>
```

```

2 #include <string>
3
4 std::vector<std::string> build_right(int n) {
5     std::vector<std::string> result;
6     result.reserve(n); // TEK tahsis: n elemanlık yer
7     for (int i = 0; i < n; ++i)
8         result.emplace_back("oge-" + std::to_string(i));
9         // emplace_back: string'i doğrudan vektörün içinde INSA eder,
10        // gecici bir string oluşturup KOPYALAMAZ (push_back bunu yapabilir)
11     return result;
12 }

```

Ölçüm

1 milyon elemanlık bir `vector<string>` için `reserve` eklemek, tipik olarak yeniden tahsis sayısını ~20'den 1'e indirir ve toplam string taşımalarını sıfıra yaklaştırır — çoğu makinede $2\times-4\times$ hızlanma. `reserve`, boyutu (size) değil kapasiteyi (capacity) değiştirir; elemanları yine `push_back`/`emplace_back` ile eklersiniz.

35.3 push_back vs emplace_back

`push_back(T(...))` önce geçici bir T nesnesi kurar, sonra onu konteynere *kopyalar* veya *taşır*. `emplace_back(...)` ise argümanları doğrudan konteynerin belleğine geçirip nesneyi *yerinde* (in place) kurar — geçici nesne ve fazladan kopya/taşıma yok.

```

1 #include <vector>
2 #include <string>
3
4 struct Point { int x, y; Point(int a, int b) : x(a), y(b) {} };
5
6 int main() {
7     std::vector<Point> pts;
8     pts.reserve(3);
9
10    pts.push_back(Point(1, 2)); // 1) gecici Point kur 2) vektöre TASI
11    pts.push_back({3, 4});     // yine gecici + tasima
12    pts.emplace_back(5, 6);    // doğrudan vektörde INSA -- gecici yok
13
14    return 0;
15 }

```

İpucu

Varsayılan olarak `emplace_back` tercih edin; en kötü ihtimalle `push_back` kadar hızlıdır. Ancak yalnızca hazır bir nesneyi ekliyorsanız (`v.push_back(std::move(mevcut))`) ikisi de aynıdır — “`emplace` her zaman daha hızlıdır” bir efsanedir, fark yalnızca *geçici nesne kurulumundan* kaçınıldığında ortaya çıkar.

35.4 Yanlış: döngüde gizli kopya (range-for)

```

1 #include <vector>
2 #include <string>
3
4 long total_length_wrong(const std::vector<std::string>& lines) {
5     long total = 0;
6     for (auto line : lines) // HATA: her eleman KOPYALANIR (string tahsisi!)

```

```

7     total += line.size();    // sadece boyutu okuyacaktik, kopya bosuna
8     return total;
9 }

```

35.5 Doğru: sabit referansla gez, kopyayı kaldır

```

1 #include <vector>
2 #include <string>
3
4 long total_length_right(const std::vector<std::string>& lines) {
5     long total = 0;
6     for (const auto& line : lines)    // const ref: KOPYA YOK, dogrudan eleman
7         total += line.size();
8     return total;
9 }

```

Dikkat

for (auto x : kap) biçimi, konteynerin her elemanını *kopyalar*. Elemanlar `std::string`, `std::vector` ya da büyük sınıflarsa bu, döngünün her turunda bir öbek tahsis + kopya demektir. Elemanı değiştirmeyecekseniz **her zaman** `const auto&`, değiştirecekseniz `auto&` kullanın. Salt `auto` yalnızca `int` gibi ucuz, kopyalanması bedava tipler için uygundur.

36. String'ler: Tahsis, Birleştirme ve string_view

String'ler, gizli tahsislerin bir numaralı kaynağıdır. Her `std::string` kısa metinler için genellikle öbek kullanmaz (aşağıdaki SSO), ama uzun metinler, birleştirmeler ve alt dizeler (`substr`) her seferinde yeni bir öbek bloğu tahsis eder.

36.1 Yanlış: operator+ zinciriyle birleştirme

```

1 #include <string>
2 #include <vector>
3
4 std::string join_wrong(const std::vector<std::string>& parts) {
5     std::string out;
6     for (const auto& p : parts)
7         out = out + p + ", ";
8     // HER turda: (out + p) yeni bir GECICI string tahsis eder,
9     // sonra (+, ") bir GECICI DAHA, sonra out'a atanır.
10    // n parca için O(n^2) karakter kopyası + 2n tahsis.
11    return out;
12 }

```

36.2 Doğru: reserve + += ile yerinde ekle

```

1 #include <string>
2 #include <vector>
3 #include <numeric>
4
5 std::string join_right(const std::vector<std::string>& parts) {
6     // 1) Toplam uzunlugu onceden hesapla, TEK tahsis yap
7     std::size_t need = 0;

```



```

8     for (const auto& p : parts) need += p.size() + 2;
9
10    std::string out;
11    out.reserve(need);           // tek seferde yeterli yer
12
13    for (const auto& p : parts) {
14        out += p;               // yerinde ekle: gecici yok
15        out += ", ";           // += operator+'nin aksine kopyalamaz
16    }
17    return out;                 // NRVO ile kopyasız doner
18 }

```

Neden bu kadar fark eder?

$a = a + b$ ifadesi a ve b 'nin toplamını tutacak *yeni* bir geçici string tahsis eder, sonra onu a 'ya atar — eski tampon çöpe gider. $a += b$ ise a 'nın mevcut tamponunda yer varsa hiç tahsis yapmadan sona ekler. Bir döngüde bu fark, $\mathcal{O}(n^2)$ 'lik bir kopya patlamasını $\mathcal{O}(n)$ 'e indirir.

36.3 Yanlış: salt okunur parametre için gereksiz kopya/tahsis

```

1  #include <string>
2  #include <cctype>
3
4  // Bu fonksiyon parametreyi degistirmez, sadece OKUR.
5  // Ama std::string alır: cagirinca cogu zaman bir KOPYA/tahsis olur.
6  bool starts_with_wrong(std::string text, char c) { // deger -> kopya
7      return !text.empty() && text.front() == c;
8  }
9  // starts_with_wrong(cok_uzun_string, 'x'); // koca string kopyalanir
10 // starts_with_wrong("literal", 'x'); // literal -> gecici string tahsisi

```

36.4 Doğru: string_view ile sahiplenmeden, tahsissiz oku

```

1  #include <string>
2  #include <string_view>
3
4  // string_view: bir karakter dizisine (isaretci + uzunluk) sahiplenmeyen
5  // bir "pencere". KOPYALAMAZ, TAHSIS ETMEZ; sadece var olan veriyi gosterir.
6  bool starts_with_right(std::string_view text, char c) {
7      return !text.empty() && text.front() == c;
8  }
9  // Hepsi TAHSISSIZ calisir:
10 // starts_with_right(cok_uzun_string, 'x'); // sadece pencere olusur
11 // starts_with_right("literal", 'x'); // literal dogrudan gorunur
12 // starts_with_right(text.substr(...) ...) // DIKKAT: substr yerine
13 // text_view.substr(...) kullan -- o da tahsissizdir

```

Dikkat

`std::string_view` güçlüdür ama **sahiplenmez**: yalnızca başka birinin belleğine bakan bir işaretçi + uzunluktur. Gösterdiği string yok olursa `string_view` *sarkan* (dangling) hâle gelir. Asla bir geçici string'in `string_view`'ını saklamayın (`std::string_view sv = get_string();` — `get_string()`'in dönüşü hemen ölür!). Salt okunur *parametreler* için idealdir; üye olarak saklarken ömrüne çok dikkat edin.

36.5 Küçük String Optimizasyonu (SSO)

```

1 #include <string>
2 #include <iostream>
3
4 int main() {
5     std::string kısa = "merhaba";           // ~15 karaktere kadar: OBEK YOK,
6                                             // karakterler string nesnesinin
7                                             // ICINDE (stack'te) saklanır -> SSO
8     std::string uzun(1000, 'x');           // uzun: OBEK tahsisi yapılır
9
10    // SSO sayesinde kısa string'ler bedava kopyalanır; bu yüzden küçük
11    // string'lerde asiri optimizasyon (mesela her yerde string_view)
12    // gereksiz olabilir -- önce OLC.
13    std::cout << kısa.capacity() << "\n"; // tipik: 15 (libstdc++)
14    return 0;
15 }
```

Bilgi

SSO (Small String Optimization): Modern standart kütüphaneler, kısa string'leri (genelde ≤ 15 karakter) öbek yerine string nesnesinin kendi içinde saklar. Bu yüzden kısa string kopyaları neredeyse bedavadır ve her string parametresini körü körüne string_view'a çevirmek her zaman kazanç sağlamaz. Kazanç, uzun string'lerde ve çok sayıda kopyada belirginleşir.

37. Taşıma Semantiğini Kaçırmak ve Yanlış std::move Kullanımı

Kısım 5'te taşıma semantiğini gördük; burada onu *optimizasyon* açısından en sık suistimal edilen iki hatayla ele alalım: (1) taşınabilecek yerde kopyalamak, (2) derleyicinin zaten yapacağı optimizasyonu std::move ile engellemek.

37.1 Yanlış: dönüşte std::move (RVO'yu bozar)

```

1 #include <vector>
2
3 std::vector<int> make_wrong() {
4     std::vector<int> v(1000, 7);
5     return std::move(v);
6     // HATA: yerel degiskeni std::move ile dondurmek, derleyicinin
7     // NRVO (Named Return Value Optimization) ile nesneyi cagiranin
8     // yerinde DOGRUDAN insha etmesini ENGELLER. std::move burada
9     // hem gereksiz hem ZARARLI: en iyi ihtimalle bir tasima ekler,
10    // RVO olsaydi SIFIR is olurdu.
11 }
```

37.2 Doğru: yerel değişkeni olduğu gibi döndür

```

1 #include <vector>
2
3 std::vector<int> make_right() {
4     std::vector<int> v(1000, 7);
5     return v;
6     // NRVO: derleyici v'yi cagiranin yerinde insha eder,
7     // hicbir kopya/tasima olmaz (copy elision)
8 }
```

Dikkat

Bir fonksiyondan *yerel bir değişkeni* döndürürken asla `return std::move(local);` yazmayın. Derleyici zaten kopya seçimini (copy elision / RVO) uygular ve nesneyi doğrudan çağırının belleğinde kurar. `std::move` eklemek bu optimizasyonu kapatır ve gereksiz bir taşıma ekler. *İstisna*: döndürdüğünüz şey bir üye ya da parametre ise (yerel değil) taşıma bazen yerinde olabilir.

37.3 Yanlış / Doğru: değere alıp saklarken

```

1 #include <string>
2 #include <vector>
3
4 class Widget {
5     std::string name_;
6     std::vector<int> data_;
7 public:
8     // YANLIŞ: const ref alıp sonra KOPYALAMAK. Çağırın gecici verse bile
9     // bu kod her zaman kopyalar.
10    // Widget(const std::string& n) : name_(n) {} // hep kopya
11
12    // DOĞRU: değere al + std::move ile ICERİ TASI.
13    // Çağırın lvalue verirse 1 kopya, rvalue (gecici) verirse 0 kopya.
14    Widget(std::string n, std::vector<int> d)
15        : name_(std::move(n)), data_(std::move(d)) {}
16    // parametreler YEREL kopyalardır; onları uyeye TASIMAK doğrudur
17    // (burada std::move doğru kullanım -- RVO devrede değil)
18 };
19
20 int main() {
21     std::vector<int> big(10000, 1);
22     Widget w("gecici", std::move(big)); // big -> parametreye tasınır,
23                                         // parametre -> uyeye tasınır. Kopya yok.
24     return 0;
25 }
```

Bilgi

“Değere al, sonra taşı” (sink parameter) deyimi: Bir kurucunun/fonksiyonun argümanı *saklayacağı* durumlarda parametreyi *değere* alıp üyeye `std::move` ile taşıyın. Bu tek imza, hem lvalue (bir kopya) hem rvalue (sıfır kopya) çağrılarında optimaldir ve `const T& + T&&` aşırı yüklemesi yazma zahmetinden kurtarır. Kural şudur: *yerel değişkeni/parametreyi* taşıyın (`std::move` doğru), *dönüş değerini* taşımayın (derleyiciye bırakın).

38. Bellek Düzeni ve Önbellek Dostu Kod

Modern işlemcilerde bir L1 önbellek erişimi ~ 1 ns iken, ana bellekten (RAM) okuma ~ 100 ns’dir — yaklaşık **100 kat** fark. Bu yüzden verinin *düzeni*, çoğu zaman algoritmanın karmaşıklığından daha çok performansı belirler. Bitişik (contiguous) ve önbellek satırına (cache line, tipik 64 bayt) sığan veri hızlıdır; parçalanmış, işaretçilerle dolaşan veri yavaştır.

38.1 Yanlış: dağınık düğümler (list / vector-of-pointers)

```

1 #include <list>
```

```

2 #include <vector>
3 #include <memory>
4
5 // std::list: her dugum AYRI bir obek tahsisi, bellekte DAGINIK.
6 // Ustunden gecmek her adimda bir onbellek iskalamasi (cache miss) demektir.
7 long sum_wrong(const std::list<int>& lst) {
8     long s = 0;
9     for (int x : lst) s += x;    // her x muhtemelen farkli bir cache line'da
10    return s;
11 }
12
13 // Ayni sorun: vector<unique_ptr<T>> -- isaretciler bitisik ama
14 // GOSTERDIKLERI nesneler obekte dagInIk.
15 long sum_ptrs_wrong(const std::vector<std::unique_ptr<int>>& v) {
16     long s = 0;
17     for (const auto& p : v) s += *p;    // her *p bir dolayli erisim + cache miss
18     return s;
19 }

```

38.2 Doğru: bitişik dizi (contiguous vector)

```

1 #include <vector>
2
3 // std::vector<int>: tum elemanlar ARDISIK bellekte. Islemci veriyi
4 // onbellek satirlari halinde onceden getirir (prefetch); dolasim cok hizli.
5 long sum_right(const std::vector<int>& v) {
6     long s = 0;
7     for (int x : v) s += x;    // ardisik erisim -> neredeyse hep cache HIT
8     return s;
9 }

```

Ölçüm

Aynı sayıda `int` için `std::vector` üzerinde toplam almak, `std::list` üzerinde almaktan tipik olarak $5 \times - 30 \times$ daha hızlıdır — algoritma ikisinde de $\mathcal{O}(n)$ olmasına rağmen. Fark tamamen bellek düzeninden ve önbellek davranışından gelir. “Ortadan sık ekleme/silme” gerçekten baskın değilse `list` yerine `vector` seçin.

38.3 AoS ve SoA: sıcak/soğuk alan ayrımı

Bir yapının yalnızca birkaç alanını sıkça işliyorsanız, tüm yapıyı (kullanılmayan alanlarıyla) ön-belleğe taşımak bant genişliğini boşa harcar. **AoS** (Array of Structs) yerine **SoA** (Struct of Arrays) düzeni, yalnızca ihtiyaç duyulan alanları bitişik tutar.

```

1 #include <vector>
2 #include <string>
3
4 // YANLIS icin de dogru icin de senaryo: 1 milyon parcacigin sadece
5 // pozisyonunu her karede guncelliyoruz; isim/renk nadiren kullaniliyor.
6
7 // --- AoS: Array of Structs (naif) ---
8 struct ParticleAoS {
9     float x, y, z;                // SICAK: her kare guncellenir (12 bayt)
10    float vx, vy, vz;            // SICAK (12 bayt)
11    std::string name;            // SOGUK: nadir kullanilir ama her nesnede DURUR
12    unsigned color;              // SOGUK

```

```

13 };
14 void update_aos(std::vector<ParticleAoS>& ps, float dt) {
15     for (auto& p : ps) { // her p ~40+ bayt -> onbellege SOGUK alanlar da gelir
16         p.x += p.vx * dt; p.y += p.vy * dt; p.z += p.vz * dt;
17     }
18 }
19
20 // --- SoA: Struct of Arrays (onbellek dostu) ---
21 struct ParticlesSoA {
22     std::vector<float> x, y, z; // SICAK alanlar ayri, BITISIK diziler
23     std::vector<float> vx, vy, vz;
24     std::vector<std::string> name; // SOGUK: ayri, sicak dongu bunlara DOKUNMAZ
25     std::vector<unsigned> color;
26 };
27 void update_soa(ParticlesSoA& ps, float dt) {
28     const std::size_t n = ps.x.size();
29     for (std::size_t i = 0; i < n; ++i) { // sadece sicak diziler taranir
30         ps.x[i] += ps.vx[i] * dt;
31         ps.y[i] += ps.vy[i] * dt;
32         ps.z[i] += ps.vz[i] * dt; // onbellek satirlari %100 verimli
33     }
34 }

```

İpucu

Sıcak/soğuk ayrımı: Bir döngü bir yapının yalnızca birkaç alanına dokunuyorsa, o alanları (sıcak) diğerlerinden (soğuk) ayırın. SoA düzeni, SIMD vektörleştirmeyi de kolaylaştırır: derleyici $x[i]$, $x[i+1]$, ... üzerinde tek talimatla çalışabilir. AoS ise kodu yazması ve “bir nesne” olarak düşünmesi daha kolaydır; ölçüm SoA’yı haklı çıkarmadıkça AoS ile başlayın.

38.4 Yapı Hizalama ve Dolgu (Padding)

Derleyici, her üyeyi kendi hizasına (alignment) yerleştirmek için üyeler arasına görünmez *dolgu* (padding) baytları ekler. Üye sırasını yanlış seçmek, yapıyı gereksiz yere büyütür — yani daha az nesne önbellege sığar.

```

1 #include <cstdint>
2 #include <iostream>
3
4 // YANLIS sıra: bol dolgu (padding)
5 struct BadLayout {
6     char a; // 1 bayt + 7 dolgu (double'i hizalamak icin)
7     double b; // 8 bayt
8     char c; // 1 bayt + 7 dolgu (yapiyi 8'e hizalamak icin)
9 }; // toplam: 24 bayt (14 bayti VERI, 10 bayti COP)
10
11 // DOGRU sıra: buyukten kucuge diz -> dolgu en aza iner
12 struct GoodLayout {
13     double b; // 8 bayt
14     char a; // 1 bayt
15     char c; // 1 bayt + 6 dolgu
16 }; // toplam: 16 bayt
17
18 int main() {
19     std::cout << sizeof(BadLayout) << "\n"; // 24
20     std::cout << sizeof(GoodLayout) << "\n"; // 16
21     // 1 milyon nesnede: 24 MB yerine 16 MB -> %33 az bellek,
22     // %33 daha fazla nesne ayni onbellege sigar.

```

```

23     return 0;
24 }

```

Kural

Yapı üyelerini genellikle **büyükten küçüğe** sıralayın (`double`/işaretçi → `int` → `short` → `char`). Bu, dolgu baytlarını en aza indirir. Boyutu doğrulamak için `sizeof` ve `alignof` kullanın; kritik veri yapılarında bu basit yeniden sıralama, bellek ayak izini görünür biçimde düşürür ve önbellek verimini artırır.

39. Tahsis Maliyetleri: `shared_ptr`, `make_shared` ve Nesne Havuzları

Öbek tahsisi (`new`/`malloc`) ucuz değildir: kilit alma, uygun blok arama, defter tutma içerir — tipik olarak yüzlerce nanosaniye. Çok sayıda küçük tahsis, hem yavaştır hem belleği parçalar (fragmentation). Bu bölümde tahsis sayısını azaltmanın yollarını görelim.

39.1 Yanlış: iki tahsis (`shared_ptr` + `new`)

```

1  #include <memory>
2  #include <string>
3
4  int main() {
5      // HATA: İKİ ayrı tahsis yapar --
6      // 1) new ile Object için obek blogu
7      // 2) shared_ptr'in kontrol blogu (referans sayaci) için AYRI blok
8      std::shared_ptr<std::string> a(new std::string("merhaba"));
9      return 0;
10 }

```

39.2 Doğru: `make_shared` (tek tahsis)

```

1  #include <memory>
2  #include <string>
3
4  int main() {
5      // DOGRU: make_shared, nesneyi VE kontrol blogunu TEK bir obek
6      // tahsisinde birlestirir. Daha hizli, daha az parcalanma, istisna guvenli.
7      auto a = std::make_shared<std::string>("merhaba");
8      return 0;
9  }

```

Dikkat

`shared_ptr`'ı gereğinden fazla kullanmak yaygın bir hatadır. Her kopya, iş parçacığı güvenliği için *atomik* bir sayaç artışı/azalışı yapar — bu, `unique_ptr`'a göre ölçülebilir bir ek yüküdür. **Varsayılanınız `unique_ptr` olsun**; sahiplik gerçekten paylaşılmıyorsa `shared_ptr`'a geçmeyin. Bir nesneye yalnızca *bakıyorsanız* (sahiplenmiyorsanız) ham işaretçi `T*` veya referans `T&` geçmek, `shared_ptr` kopyalamaktan hem daha hızlı hem daha dürüst bir niyet ifadesidir.

39.3 Yanlış: sıcak döngüde binlerce küçük tahsis

```

1 #include <vector>
2 #include <memory>
3
4 struct Particle { float x, y, life; };
5
6 void simulate_wrong(int frames) {
7     for (int f = 0; f < frames; ++f) {
8         std::vector<std::unique_ptr<Particle>> ps;
9         for (int i = 0; i < 10000; ++i)
10             ps.push_back(std::make_unique<Particle>()); // her kare 10.000 NEW
11         // ... isle ...
12     } // her kare 10.000 tahsis + 10.000 serbest birakma -> cok yavas
13 }

```

39.4 Doğru: tamponu yeniden kullan (havuz / arena)

```

1 #include <vector>
2
3 struct Particle { float x, y, life; };
4
5 void simulate_right(int frames) {
6     std::vector<Particle> ps; // TEK, yeniden kullanılabilir tampon
7     ps.reserve(10000); // bir kez tahsis et
8
9     for (int f = 0; f < frames; ++f) {
10         ps.clear(); // kapasiteyi KORUR, belleği serbest BIRAKMAZ
11         for (int i = 0; i < 10000; ++i)
12             ps.push_back({0.f, 0.f, 1.f}); // tahsis YOK, mevcut yere yaz
13         // ... isle ...
14     } // tum simulasyon boyunca yalnızca 1 (belki birkaç) tahsis
15 }

```

Basit nesne havuzu

Nesneleri değil de *tamponu* yeniden kullanmak (yukarıdaki clear + yeniden doldurma) en basit ve en etkili havuz (pool) tekniğidir. Daha karmaşık senaryolarda (nesnelerin ömürleri serpiştirilmişse) bir *serbest liste* (free list) ya da *arena/bump allocator* kullanılır: büyük bir blok bir kez ayrılır, tahsisler yalnızca bir işaretçiyi ilerletir, hepsi birlikte serbest bırakılır. Amaç hep aynıdır: **tahsis sayısını azaltmak**.

```

1 #include <vector>
2 #include <cstdint>
3
4 // Çok basit bir "bump/arena" ayırıcı: tek blok, tahsis = isaretci ilerlet.
5 class Arena {
6     std::vector<std::byte> buffer_;
7     std::size_t offset_ = 0;
8 public:
9     explicit Arena(std::size_t bytes) : buffer_(bytes) {}
10
11     void* allocate(std::size_t n, std::size_t align = alignof(std::max_align_t)) {
12         std::size_t p = (offset_ + align - 1) & ~(align - 1); // hizala
13         if (p + n > buffer_.size()) return nullptr; // dolu
14         offset_ = p + n;
15         return buffer_.data() + p; // tahsis = sadece offset ilerletmek
16     }

```

```

17 void reset() { offset_ = 0; } // TUMUNU birden "serbest bırak" (O(1))
18 };
19
20 int main() {
21     Arena arena(1 << 20); // 1 MB'lık tek blok
22     int* a = static_cast<int*>(arena.allocate(sizeof(int)));
23     int* b = static_cast<int*>(arena.allocate(sizeof(int)));
24     if (a && b) { *a = 1; *b = 2; }
25     arena.reset(); // tüm tahsisler tek hamlede geçersiz
26     return 0;
27 }

```

İpucu

Arena/bump ayırıcı, “aynı anda oluşturulup aynı anda yok edilen” çok sayıda kısa ömürlü nesne için idealdir (bir kare, bir istek, bir ayırıştırma turu). Tahsis neredeyse bedavadır (bir toplama işlemi) ve serbest bırakma $O(1)$ 'dir (offset'i sıfırla). Standart kütüphanede `std::pmr::monotonic_buffer_resource` bu kalıbı hazır sunar ve `std::pmr` konteynerleriyle doğrudan kullanılabilir.

40. İşaretçi ve Kaynak Yönetimi: Klasik Hatalar

Performansın altında bir de *doğruluk* katmanı vardır: işaretçilerle yapılan hatalar (sarkan işaretçi, iki kez silme, geçersiz kılınmış iteratör) programı yavaşlatmaz — *çökertir* veya sessizce belleği bozar. Bu hatalar en sık ham (`new/delete`) bellek yönetiminde ve konteyner yeniden tahsislerinde ortaya çıkar.

40.1 Yanlış: yerel değişkenin adresini döndürmek

```

1 #include <string>
2
3 // HATA: 'local' fonksiyon bitince yığın (stack) üzerinde YOK OLUR;
4 // donen isaretci SARKAN (dangling) olur -> use-after-return (UB).
5 const std::string* name_wrong() {
6     std::string local = "gecici";
7     return &local; // local birazdan olacak!
8 }
9 // int& ref_wrong() { int x = 5; return x; } // ayni hata, referansla

```

40.2 Doğru: değeri döndür (ya da ömrü çağırana ait olsun)

```

1 #include <string>
2 #include <memory>
3
4 // 1) En basiti: DEGERI dondur (NRVO ile kopyasiz gelir)
5 std::string name_right() {
6     std::string local = "gecici";
7     return local; // nesnenin OMRU cagirana tasinir
8 }
9
10 // 2) Obekte yasamasi gerekiyorsa sahipligi acikca devret
11 std::unique_ptr<std::string> make_name() {
12     return std::make_unique<std::string>("obekte");
13 }

```



```
13 } // sahiplik cagirana gecer; kimse unutamaz, kimse iki kez silemez
```

Dikkat

Yerel bir nesnenin adresini/referansını asla döndürmeyin. Fonksiyon dönünce yığın çerçevesi (stack frame) geçersiz olur; o adrese erişim tanımsız davranıştır (use-after-return) ve genelde “bazen çalışıp bazen çökme” gibi bulunması en zor hatalara yol açar. Derleyicinin -Wreturn-local-addr (GCC/Clang) uyarısını ciddiye alın; -fsanitize=address (AddressSanitizer) bu hataları çalışma zamanında kesin olarak yakalar.

40.3 Yanlış: yeniden tahsis iteratörü/işaretçiyi geçersiz kılar

```
1 #include <vector>
2 #include <iostream>
3
4 int main() {
5     std::vector<int> v{1, 2, 3};
6     int* p = &v[0];           // ilk elemani gosteren ham isaretci
7
8     for (int i = 0; i < 1000; ++i)
9         v.push_back(i);       // bir noktada vektor YENIDEN TAHSIS edilir:
10                                // tum elemanlar YENI bloğa tasınır, ESKI blok
11                                // serbest bırakılır -> p artık SARKAN!
12
13     std::cout << *p << "\n";  // UB: gecersiz (silinmiş) belleğe erişim
14     return 0;
15 }
```

40.4 Doğru: yeniden tahsisi önle veya indeksle çalış

```
1 #include <vector>
2 #include <iostream>
3
4 int main() {
5     std::vector<int> v{1, 2, 3};
6     v.reserve(1003);          // 1) tahsisi ONCEDEN yap -> blok tasınmaz
7
8     int* p = &v[0];           // reserve'den SONRA al -> geçerli kalır
9     for (int i = 0; i < 1000; ++i)
10         v.push_back(i);
11
12     std::cout << *p << "\n";  // güvenli: blok yerinde kaldı
13
14     // 2) Daha güvenlisi: ham isaretci yerine INDEKS sakla.
15     std::size_t idx = 0;      // indeks yeniden tahsisten ETKİLENMEZ
16     std::cout << v[idx] << "\n";
17     return 0;
18 }
```

Geçersizleşme kuralı

`std::vector`'e ekleme yapmak (`push_back`, `insert`) yeniden tahsis tetiklerse, o vektöre ait *tüm* işaretçiler, referanslar ve iteratörler geçersizleşir. `erase` ise silinen konumdan sonrasını geçersiz kılar. Uzun ömürlü bir tutamağa (*handle*) ihtiyacınız varsa ham işaretçi yerine **indeks** saklayın ya da elemanları `std::deque`/`std::list` gibi ekleme sırasında mevcut elemanları taşımayan bir konteynerde tutun.

40.5 Yanlış: ham işaretçiyle belirsiz sahiplik (çift silme / sızıntı)

```

1 #include <string>
2
3 void raw_wrong() {
4     std::string* s = new std::string("veri");
5     std::string* alias = s;      // kim sahip? BELIRSIZ.
6     delete s;                  // s silindi
7     delete alias;              // AYNI bellek IKINCI kez silindi -> UB (double free)
8
9     std::string* leak = new std::string("kayıp");
10    // ... erken return / istisna ... delete unutuldu -> SIZINTI
11 }
```

40.6 Doğru: sahipliği `unique_ptr` ile tek ve net yap

```

1 #include <string>
2 #include <memory>
3
4 void raw_right() {
5     auto s = std::make_unique<std::string>("veri");
6     std::string* peek = s.get(); // GOZLEM için ham isaretci (sahiplenmez)
7     // 'peek' asla delete EDILMEZ -- sahibi s'dir, o da otomatik siler.
8
9     // Sahipligi devretmek gerekirse: acikca TASI
10    std::unique_ptr<std::string> other = std::move(s);
11    // s artık bos; iki kez silme imkansız, istisna olsa bile sızıntı yok.
12 }
```

Bilgi

Sahiplik/gözlem ayrımı: Bir işaretçi ya bir kaynağın *sahibidir* (onu silmekle yükümlü) ya da ona yalnızca *bakar* (gözlemci). Sahipliği `unique_ptr` (tekil) veya `shared_ptr` (paylaşımlı) ile ifade edin; gözlem için çıplak `T*` veya `T&` kullanın ama onları asla `delete` etmeyin. Bu ayrımı tipe kodladığınızda çift silme ve sızıntı *sınıfsal* olarak imkânsızlaşır — “kim siliyor?” sorusu bir daha sorulmaz.

41. Beşin Kuralı: Kaynak Yöneten Sınıfı Doğru Yazmak

Kısım 5'te Beşin Kuralı'nı ilkece gördük; burada onu *bellek yönetimi hatası* açısından ele alalım. Bir sınıf *ham* bir kaynağı (öbek tamponu, dosya, soket) elle yönetiyorsa, derleyicinin ürettiği *varsayılan* kopya/taşıma işlemleri o kaynağı **sığ** (shallow) kopyalar — ve bu neredeyse her zaman çift silmeye veya sızıntıya yol açar.

41.1 Yanlış: yalnızca yıkıcı (Üçün Kuralı ihlali)

```

1  #include <cstdint>
2  #include <algorithm>
3
4  class Buffer {
5      int*      data_;
6      std::size_t size_;
7  public:
8      explicit Buffer(std::size_t n) : data_(new int[n]), size_(n) {}
9      ~Buffer() { delete[] data_; }           // yıkıcı VAR ama kopya/tasima YOK
10
11      // Derleyici VARSAYILAN kopya yapıcıyı üretir: SIG kopya!
12      // b2, b1'in data_ ISARETCISINI kopyalar -> IKISI de aynı bloga bakar.
13 };
14
15 void bug() {
16     Buffer b1(100);
17     Buffer b2 = b1;    // sig kopya: b2.data_ == b1.data_
18 }                    // once b2 yıkılır: delete[] data_
19                    // sonra b1 yıkılır: AYNI data_ TEKRAR delete[] -> UB!

```

41.2 Doğru: beş özel fonksiyonu da tanımla

```

1  #include <cstdint>
2  #include <algorithm>
3  #include <utility>
4
5  class Buffer {
6      int*      data_;
7      std::size_t size_;
8  public:
9      explicit Buffer(std::size_t n) : data_(new int[n]{}), size_(n) {}
10
11      // 1) YIKICI
12      ~Buffer() { delete[] data_; }
13
14      // 2) KOPYA YAPICI: DERİN kopya -- yeni blok ayır, içerigi kopyala
15      Buffer(const Buffer& o) : data_(new int[o.size_]), size_(o.size_) {
16          std::copy(o.data_, o.data_ + size_, data_);
17      }
18
19      // 3) KOPYA ATAMA: copy-and-swap ile (self-atamaya ve istisnaya güvenli)
20      Buffer& operator=(const Buffer& o) {
21          Buffer tmp(o);           // önce kopyala
22          swap(tmp);              // sonra takas et -> eski kaynak tmp ile silinir
23          return *this;
24      }
25
26      // 4) TASIMA YAPICI: kaynaklı CAL, otekini boşalt (noexcept önemli!)
27      Buffer(Buffer&& o) noexcept : data_(o.data_), size_(o.size_) {
28          o.data_ = nullptr;      // o artık silecek bir şey sahiplenmiyor
29          o.size_ = 0;
30      }
31
32      // 5) TASIMA ATAMA: mevcut kaynaklı bırak, otekininkini cal
33      Buffer& operator=(Buffer&& o) noexcept {
34          if (this != &o) {       // self-tasimaya karsi koru
35              delete[] data_;    // kendi eski kaynakımı bırak

```

```

36         data_ = o.data_; size_ = o.size_;
37         o.data_ = nullptr; o.size_ = 0;
38     }
39     return *this;
40 }
41
42 void swap(Buffer& o) noexcept {
43     std::swap(data_, o.data_);
44     std::swap(size_, o.size_);
45 }
46 std::size_t size() const { return size_; }
47 };

```

Dikkat

Üçün/Beşin Kuralı: Bir sınıf yıkıcı, kopya yapıcı veya kopya atama operatöründen *herhangi birini* elle tanımlıyorsa, büyük olasılıkla *beşini de* (artı taşıma yapıcı ve taşıma atama) doğru tanımlamalıdır. Yalnızca yıkıcı yazıp gerisini derleyiciye bırakmak, sık kopya yüzünden çift silmeye yol açar. Taşıma işlemlerini noexcept yapmayı **unutmayın**: `std::vector` gibi konteynerler, yeniden tahsis sırasında yalnızca *noexcept* taşımayı kullanır; aksi halde güvenlik için *kopyalamaya* düşerler ve tüm optimizasyonu kaybedersiniz.

41.3 Doğru (daha iyisi): Sıfırın Kuralı

Yukarıdaki beş fonksiyonu *elle* yazmak hataya açıktır. En iyi çözüm, kaynağı zaten kendini yöneten bir üyeye (`std::vector`, `std::unique_ptr`, `std::string`) devredip *hiçbir* özel fonksiyon yazmamaktır — buna **Sıfırın Kuralı** (Rule of Zero) denir.

```

1  #include <vector>
2  #include <cstdint>
3
4  // Ham int* yerine std::vector kullan: kopya, tasima ve yikim
5  // DOGRU sekilde OTOMATIK gelir. Bes fonksiyonun HICBIRINI yazmayiz.
6  class Buffer {
7      std::vector<int> data_;          // kaynaklari kendisi yönetir (RAII)
8  public:
9      explicit Buffer(std::size_t n) : data_(n) {}
10     std::size_t size() const { return data_.size(); }
11     // ~Buffer, kopya, tasima: HEPSI derleyiciden, hepsi DOGRU.
12 };
13
14 int main() {
15     Buffer a(100);
16     Buffer b = a;                // derin kopya -- vector kopyalar (dogru)
17     Buffer c = std::move(a);     // tasima -- vector tasinir (ucuz, dogru)
18     return 0;
19 }

```

İpucu

Önce Sıfırın Kuralı'nı hedefleyin. Sınıfınızın üyeleri kendi kaynaklarını yönetiyorsa (akıllı işaretçiler ve standart konteynerler), hiçbir özel üye fonksiyon yazmanıza gerek kalmaz; derleyicinin ürettikleri zaten doğrudur. Beşin Kuralı'nı yalnızca gerçekten *ham* bir kaynağı (bir C API tanıtıcısı, elle new'lenen bellek) sarmaladığınız — ve bunu tek bir sınıfta izole ettiğiniz — ender durumlarda uygulayın. Modern C++'ta “çıplak new/delete neredeyse hiç görünmez” kuralının pratik karşılığı budur.

Kural	Ne zaman
Sıfırın Kuralı	Üyeler kendini yönetiyor (<code>vector</code> , <code>unique_ptr</code> , <code>string</code>). Hiçbir özel fonksiyon yazma. <i>Varsayılan tercih.</i>
Üçün Kuralı	Ham kaynak var; en az yıkıcı+kopya yapıcı+kopya atama gerekli (C++98 mirası).
Beşin Kuralı	Ham kaynak var; performans için taşıma yapıcı+taşıma atama da (<code>noexcept</code>) eklenir.

42. Özet: Bellek Optimizasyonu Kontrol Listesi

Yanlış (yaygın hata)	Doğru (tercih)
<code>reserve</code> 'süz döngüde <code>push_back</code>	Boyut biliniyorsa önce <code>reserve(n)</code>
<code>for (auto x : kap) (kopya)</code>	<code>for (const auto& x : kap)</code>
Geçici kurup <code>push_back</code>	<code>emplace_back(args...)</code>
<code>out = out + p</code> döngüde	<code>out.reserve(...)</code> + <code>out += p</code>
Salt okunur <code>std::string</code> parametresi	<code>std::string_view</code> (ömre dikkat)
<code>return std::move(local)</code>	<code>return local</code> (RVO'ya bırak)
Saklamak için <code>const T&</code> alıp kopyalamak	Değere al + <code>std::move</code> (sink)
<code>std::list</code> / <code>vector<ptr></code> tarama	Bitişik <code>std::vector<T></code>
Tüm alanları her döngüde gezmek	Sıcak/soğuk ayrımı (SoA)
Rastgele üye sırası (bol dolgu)	Büyükten küçüğe sırala
<code>shared_ptr(new T)</code>	<code>make_shared<T>(...)</code>
Her yerde <code>shared_ptr</code>	Varsayılan <code>unique_ptr</code> ; gözlem için <code>T*/T&</code>
Sıcak döngüde bin küçük <code>new</code>	Tamponu yeniden kullan / arena
Yerel değişkenin adresini döndürmek	Değeri döndür / <code>unique_ptr</code> ile sahiplik devret
Yeniden tahsis sonrası eski işaretçi/iteratör	Önce <code>reserve</code> ya da indeks sakla
Ham işaretçide belirsiz sahiplik (double free)	<code>unique_ptr</code> ; gözlem için <code>.get()</code>
Ham kaynakta yalnızca yıkıcı	Beşin Kuralı (5 fonksiyon, <code>noexcept</code> taşıma)
Beş fonksiyonu elle yazmak	Sıfırın Kuralı: kaynağı üye tipe devret

Bilgi

Öncelik sırası: (1) Gereksiz *tahsisleri* yok edin — en büyük kazanç genelde buradadır. (2) Gereksiz *kopyaları* yok edin (referans, taşıma, `string_view`). (3) Veriyi *bitişik* ve *önbellek dostu* düzenleyin. (4) Yalnızca ölçüm gösterirse mikro optimizasyona (hizalama, SoA, özel ayırıcı) inin. Her adımda profilleyiciyle *öncesi/sonrası* ölçün — optimizasyon bir inanç değil, bir ölçüm disiplini-
dir.

KISIM 13

Modern C++ (C++17 / C++20)

43. Yapısal Bağlama, if-init ve Yararlı Tipler

Modern C++, günlük kodu belirgin biçimde kısaltan ve güvenliştiren pek çok özellik getirdi. Bunlardan bazılarını (auto, range-for) zaten kullandık; burada en pratik olanları toplu görelim.

```

1  #include <iostream>
2  #include <map>
3  #include <tuple>
4  #include <optional>
5  #include <string>
6  #include <string_view>
7
8  // std::optional: "değer olmayabilir" durumunu tip güvenli ifade et
9  std::optional<int> parsePositive(int x) {
10     if (x > 0) return x;           // değer var
11     return std::nullopt;          // değer yok
12 }
13
14 // string_view: kopyalamadan string'e salt-okunur bakış (ucuz)
15 void printLength(std::string_view sv) { // ne string kopyası ne const char*
16     std::cout << "\"" << sv << "\" -> " << sv.size() << " karakter\n";
17 }
18
19 int main() {
20     // YAPISAL BAĞLAMA (C++17): pair/tuple/struct'i tek satırda parçala
21     std::map<std::string, int> scores{{"Ada", 90}, {"Alan", 85}};
22     for (const auto& [name, score] : scores) // .first/.second yerine
23         std::cout << name << " = " << score << "\n";
24
25     std::tuple<int, double, std::string> record{1, 3.14, "pi"};
26     auto [id, value, label] = record;        // ucunu birden al
27     std::cout << id << " " << value << " " << label << "\n";
28
29     // IF-INIT (C++17): koşul içinde değişken tanımla (kapsamı dar tut)
30     if (auto result = parsePositive(42); result.has_value())
31         std::cout << "Pozitif: " << *result << "\n";
32
33     if (auto result = parsePositive(-5); !result)
34         std::cout << "Değer yok, varsayılan: " << result.value_or(0) << "\n";
35
36     // string_view: gecici string, string literal, hepsi kopyasız çalışır

```

```

37     std::string s = "merhaba dünya";
38     printLength(s);
39     printLength("sabit metin");
40     return 0;
41 }

```

İpucu

`std::string_view`, bir string'i *kopyalamadan* okumak için idealdir; fonksiyon parametrelerinde `const std::string&` yerine sıkça tercih edilir çünkü string literalleri ve alt-dizeleri de kopyasız kabul eder. Ancak dikkat: `string_view` veriye *sahip değildir*; gösterdiği string'ten daha uzun yaşarsa sarkan görünüm oluşur. Sahiplik gerekiyorsa `std::string` kullanın.

44. Concepts: Şablonlara Kısıt (C++20)

C++20 *concepts*, şablon parametrelerine *kısıtlar* koyar: “bu şablon yalnızca sayısal tiplerle çalışır” gibi. Bu, hem niyeti belgeler hem de yanlış tip kullanıldığında okunması imkânsız şablon hataları yerine *net* hata mesajları verir.

```

1  #include <iostream>
2  #include <concepts>
3
4  // Concept tanımı: T aritmetik (sayısal) bir tip olmalı
5  template <typename T>
6  concept Numeric = std::integral<T> || std::floating_point<T>;
7
8  // Yalnızca Numeric tipleri kabul eden şablon
9  template <Numeric T>
10 T square(T x) {
11     return x * x;
12 }
13
14 // Alternatif kısıt sozdizimi: requires ile
15 template <typename T>
16     requires std::floating_point<T>
17 T average(T a, T b) {
18     return (a + b) / 2;
19 }
20
21 int main() {
22     std::cout << square(5) << "\n";           // int -> 25
23     std::cout << square(2.5) << "\n";         // double -> 6.25
24     // square("metin"); // DERLEME HATASI: net mesaj -> "Numeric sağlanmadı"
25
26     std::cout << average(3.0, 5.0) << "\n";   // 4.0
27     return 0;
28 }

```

```

1  g++ -std=c++20 concepts.cpp -o concepts

```


Bilgi

Concepts, C++ şablon programlamanın en büyük kalite sıçramalarından biridir. `Numeric`, `std::integral`, `std::floating_point`, `std::sortable` gibi kısıtlar sayesinde derleyici, bir şablonu yanlış tiple kullandığımızda *tam olarak hangi kısıtın sağlanmadığını* söyler — eski günlerin yüzlerce satırlık anlaşılmaz şablon hatalarına son verir.

45. Ranges: Bileştirilebilir Algoritmalar (C++20)

C++20 *ranges* kütüphanesi, STL algoritmalarını hem daha kısa (`begin()/end()` yazmadan) hem de *bileştirilebilir* (composable) hale getirir. “Görünümler” (views) sayesinde filtreleme, dönüştürme gibi işlemleri bir boru hattı (pipeline) gibi zincirleyebilirsiniz — üstelik tembel (lazy) değerlendirmeyle.

```

1  #include <iostream>
2  #include <vector>
3  #include <ranges>
4  #include <algorithm>
5
6  int main() {
7      std::vector<int> numbers{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
8
9      // Klasik algoritma: begin()/end() gerekmez
10     std::ranges::sort(numbers);
11
12     // Boru hattı: çift sayıları al -> karesini al (tembel değerlendirme)
13     auto pipeline = numbers
14         | std::views::filter([](int x){ return x % 2 == 0; }) // 2,4,6,8,10
15         | std::views::transform([](int x){ return x * x; });   // 4,16,36,64,100
16
17     std::cout << "Çift kareler: ";
18     for (int x : pipeline) std::cout << x << " ";
19     std::cout << "\n";
20
21     // views::take, views::reverse gibi baska gorunumler
22     std::cout << "İlk 3 (tersten): ";
23     for (int x : numbers | std::views::reverse | std::views::take(3))
24         std::cout << x << " "; // 10 9 8
25     std::cout << "\n";
26     return 0;
27 }
```

```
1 g++ -std=c++20 ranges.cpp -o ranges
```

İpucu

Ranges’ın | boru hattı sözdizimi, veri dönüşümlerini son derece okunur kılar: “filtrele, sonra dönüştür, sonra ilk 3’ünü al” niyetini doğrudan ifade eder. Görünümler *tembeldir*: ara diziler oluşturmaz, elemanlar yalnızca gezinirken hesaplanır — bu hem bellek hem hız açısından verimlidir. Modern C++ kodunda döngü ve ara vektör yığınlarının yerini giderek ranges alıyor.

46. Kapanış ve Altın Kurallar

Bu rehber, C++'ı temellerinden modern özelliklerine kadar kapsadı: sözdizimi, nesne yönelimi, bellek ve kaynak yönetimi, şablonlar, STL, lambda, istisnalar ve C++20 yenilikleri. Aşağıda, özellikle kaynak yönetimi ve sınıf tasarımı için özet altın kurallar — günlük kodda en çok fark yaratanlar:

Altın Kural

- 1 **Sıfır Kuralını** hedefleyin: kaynağı `unique_ptr`/konteynerlere sarın, özel üye fonksiyonları hiç yazmayın.
- 2 Çıplak `new/delete` kullanmayın; `make_unique/make_shared` tercih edin.
- 3 **Varsayılan olarak `unique_ptr`**; sahiplik gerçekten paylaşılmalıysa `shared_ptr`; döngüyü kırmak için `weak_ptr`.
- 4 Beşin Kuralını uygularsanız *beşini de* bilinçli tanımlayın; taşımaları `noexcept` yapın.
- 5 Kopya atama için **copy-and-swap** deyimini kullanın.
- 6 Tek argümanlı yapıcılar ve `operator bool`'u `explicit` yapın.
- 7 Durumu değiştirmeyen üye fonksiyonları `const` yapın (`const-correctness`).
- 8 `nullptr` kullanın (`NULL/0` değil); sildikten sonra işaretçiyi `nullptr`'a atayın.
- 9 Operatörleri sezgisel anlamlarına sadık kalarak aşırı yükleyin; `operator+`'yı `operator+=` cinsinden yazın.
- 10 Derleyici uyarılarını açın: `-Wall -Wextra`; bellek hataları için `-fsanitize=address`.

Bilgi

Bellek hatalarını yakalama: Bu rehberdeki manuel kaynak yöneten örnekleri test ederken AddressSanitizer'ı kullanın; sızıntıları, sarkan işaretçileri ve ikili silmeleri anında yakalar:

```

1 # AddressSanitizer + tüm uyarılar ile derleme (gelistirme sırasında)
2 g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined \
3     program.cpp -o program
4 ./program          # bellek hatalari calisirken raporlanIr
5
6 # Valgrind ile sızıntı kontrolü (alternatif)
7 g++ -std=c++20 -g program.cpp -o program
8 valgrind --leak-check=full ./program

```

Rehberin Sonu

C++17/20 • RAII • Beşin & Sıfırın Kuralı • İyi kodlamalar!